An Algebraic Map of HAETAE-2 Implementation Verification

A Companion Guide for Pure Algebraists and Algebraic Geometers

HAETAE top-down EasyCrypt working document

Status as of: 13 August 2026 (Post-PAPER–FREEZE faithful augmented KeyGen coefficient-product composition)

Abstract

HAETAE-2 is a module-lattice signature over $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$. The objects actually proved about in this repository, however, extend beyond the ring itself. Elements and module maps from the paper are represented as canonical byte strings, those strings are placed in finite memory, and the proofs descend to the fact that actual generated Jasmin procedures preserve or invert those representations. This guide organizes that process into three layers: algebraic objects, representation maps, and state-machine refinement.

At the snapshot serving as the latest audit baseline, 82 authored EasyCrypt targets were included in the frozen manifest and passed an aggregate `-no-eco` fresh compile. Post-freeze standalone theories are not included in that count. After the 6 Week 10 rANS encoder targets came 7 Week 11 encoder-closure targets, 8 Week 12 decoder semantic-refinement targets, 2 Week 13 actual-core-composition targets, 5 Week 14 production full-HBZ boundary targets, 2 Week 15 fixed all-6 success targets, and 8 Week 16 MINCORE KeyGen/KG–NTT boundary and Sign/Verify control/word targets. The completed core covers the secret key's 992-byte verification-key prefix, agreement of the 32-byte μ between raw/internal Sign and Verify, challenge-suffix absorption, the signature-prefix round trip, HBZ coefficient–symbol leaf inverses, certificates for the actual 13-symbol rANS table, pure rANS step and byte-stack theorems, actual suffix copying, encoder–decoder harness control, and the exact equality between the entire successful output of the actual encoder and pure TraceBytes(SymbolList($symbols_0$)). It also includes refinement of the actual decoder recovering 1024 symbols from an exact trace, the core inverse composing those two theorems and actual suffix copying in `Mode2RansActualHarness.run`, and the production full-HBZ wrapper inverse lifting actual `_encode_hb_z1_full`/`_decode_hb_z1_full` to the SignaturePack/Unpack boundary. Week 15 excludes actual failure for the all-6 stream generated by canonical all-zero HBZ and proves $bad = 0$, $4 \le size \le 1024$, exact trace/frame, and the production zero round trip for executions that return. This is fixed-input Hoare partial correctness, not termination, losslessness, or probability-1 success.

For a snapshot that calls actual `_kp_m23_matrix` and `_keypair_finalize_m23` consecutively, Week 16 established $r = 2b_1 + b_0$, $b_0 \in [-1,1]$, adjusted $s_2 = s_2 - b_0$, and a snapshot-only zero congruence mod $2q$. However, even after actual output_row_from_mode2_ntt_words fixed `output_row` to the actual full-invNTT/pointwise representation, the full-NTT convolution theorem and representation adapter needed to transfer it to a native R_q product and the security model's $A_{\text{gen}}s_{\text{gen}}$ product were absent at the frozen-paper stage. KeyGen was therefore frozen at KG-2/finalization under `STOP-KG-NTT`. Direct actual-call/control results for the accepted Sign core then compiled, and a partial MINCORE-VERIFY theorem was added at the canonical decoded (x, v, h, c) boundary: V-1/V-2/V-5/V-6, the W64 norm gate, and tail-trace/mismatch word expressions remain proved, with explicit matrix and challenge blockers. The frozen-paper verdict is `STOP-VERIFY-MATRIX-CRT`.

Post-freeze proofs completed the pure full-NTT spectral action, native R_q row product, and R_q–HAETAE dot-product representation transport, and interpreted actual `avec` expansion as the paper's a. The paper's qj term was independently composed into the snapshot congruence. However, the existing security-side `haetae_mode23_qj_vector` is a synthetic generator, and concrete coefficients were proved to differ from the paper's qj and actual SHAKE128/rejection expansion. The current verdict is `STOP-KG-AVEC-QJ`, with **OBL-KG-SECURITY-EXPANDVECA-SEMANTICS** as the first repair obligation. The second h suffix codec, complete signature codec, product/replay transport connecting stored μ in completed API traces, full functional correctness, and implementation-level security also remain open. This guide does not declare a complete proof; it marks exactly which parts of the commuting diagram have been closed.

The latest post-freeze `KgFaithfulAugmentedModel.ec` also passes standalone fresh compilation. It instantiates a width-6 carrier with the actual matrix/secret and defines a structured coefficient product by transporting the generated 3-term row product to HAETAE `poly_dot`. actual_faithful_augmented_productE expands this product on active rows, while checked_mode2_parent_m23_finalize_faithful_augmented_paper_as_qj shows, at every active coefficient, its congruence with faithful (qj) mod-(2q). The product takes only the matrix and secret as inputs, reconstructs the middle 2-by-3 block from their columns, and evaluates it with `poly_dot`. The verdict for this work is therefore `GO-KG-FAITHFUL-AUGMENTED`. Its mod-$2q$ block-specific semantics nevertheless differ from the existing mod-q, width-4 security `matrix_vec_mul`; agreement with the synthetic security object is not a conclusion. This separate closure therefore changes neither the frozen target count nor the current security gate.

One-sentence summary

The current results establish that several deterministic representation squares of the signature scheme commute down to actual generated code; they do not mean that the entire signature scheme over the ring has already been verified. The next central gate is **OBL-KG-SECURITY-EXPANDVECA-SEMANTICS**, replacing/refining synthetic security ExpandVecA to agree with actual SHAKE128/rejection semantics. Week 15's fixed all-6 failure exclusion is **PROVED** but gives no termination result. The KG-2-like decomposition, snapshot congruence, and final NTT-word representation rewrite at Week 16's actual KeyGen snapshot/finalizer boundary are complete. The full-NTT convolution/representation adapter missing from the frozen paper was completed post-freeze, but revealed a later obstacle: actual `avec`, the paper's qj, and the synthetic security object cannot be identified. The current verdict is therefore `STOP-KG-AVEC-QJ`, without overwriting the frozen-paper verdict `STOP-VERIFY-MATRIX-CRT`. This is not a verdict of completed full KeyGen, h codec, full signature codec, or Sign–Verify correctness.

Contents

1 Reading contract and overall map **6**

1.1 What this guide assumes and explains 6

1.2 Status terms include logical quantifiers 6

1.3 How far algebraic geometry applies 7

1.4 Six statements to remember first 7

2 Algebraic objects: from quotient rings to signature bytes **8**

2.1 The base rings and mode 2 parameters 8

2.2 Viewing KeyGen as a module map 9

2.3 The Week 16 KeyGen snapshot congruence . . . 9
2.4 The transcript where Sign and Verify meet . . . 10
2.5 Signature representation is a partial isomorphism on a canonical subset . . . 11
2.6 Which algebraic facts belong to which layers . . . 11

3 A dictionary for representation maps and program logic 12
3.1 Arrays and memory are different objects . . . 12
3.2 Two address representations and the canonical range . . . 12
3.3 A procedure is a partial state transition, not a total function . . . 13
3.4 Refinement is a commutative square . . . 13
3.5 A frame is a theorem about write support . . . 14
3.6 Canonicality chooses representatives . . . 14
3.7 Exact traces and ghost observations . . . 14
3.8 The exact scope of "zero-loss" . . . 15
3.9 Reading one EasyCrypt theorem as a mathematical statement . . . 15

4 The proof chain currently completed 15
4.1 Three main lines of progress . . . 15
4.2 Stage 1: the verification-key quotient of a packed secret key . . . 16
4.3 Stage 2: from local arrays to public memory . . . 16
4.4 Stage 3: generated raw μ for Sign and Verify . . . 17
4.5 Stage 4: from whole-memory equality to read-region equality . . . 18
4.6 Stage 5: the position 64 challenge suffix . . . 18
4.7 Stage 6: actual caller control and accepted Verify . . . 18
4.8 Stage 7: section–retraction for the actual signature prefix . . . 19
4.9 Stage 8: the HBZ coefficient–symbol leaf . . . 19
4.10 Stage 9: actual tables and pure rANS mathematics . . . 19
4.11 Stage 10: local semantics of the actual encoder . . . 20
4.12 Stage 11: global trace closure of the actual encoder . . . 21
4.13 Stage 12: exact-trace semantics of the actual decoder . . . 21
4.14 Stage 13: the actual encoder–copy–decoder core inverse . . . 22
4.15 Stage 14: the production full-HBZ wrapper boundary . . . 23
4.16 Stage 15: fixed all-6 actual-success postconditions . . . 24
4.17 Stage 16: the frozen actual KeyGen snapshot and KG–NTT stopping boundary . . . 24
4.18 Stage 17: post-freeze NTT/Rq transport and the actual avec/qj boundary . . . 25
4.19 Exact endpoints of the completed chains . . . 26

5 The HBZ/rANS dynamical system and fixed-input failure exclusion 28
5.1 Why success postconditions and termination are different claims . . . 28
5.2 The 13-symbol partition . . . 28
5.3 The pure step is an inverse of Euclidean division . . . 29
5.4 Normalization bytes are coordinates of the discarded quotient . . . 29
5.5 A pure trace for the complete symbol list . . . 30
5.6 The six encoder targets audited in Week 10 . . . 30
5.7 The actual encoder invariant closed in Week 11 . . . 31
5.8 The actual decoder invariant closed in Week 12 . . . 31

5.9 Actual harness core composition closed in Week 13 . . . 32
5.10 Correctly reading the full HBZ wrapper closed in Week 14 . . . 33
5.11 Week 15's fixed all-6 budget and actual failure exclusion . . . 33

6 Open boundaries and limits on claims 35
6.1 The narrow but important semantic gap: product/replay . . . 35
6.2 Accepted-path and legitimate-signature correctness run in different directions . . . 36
6.3 Representation obligations remaining in the full signature codec . . . 36
6.4 Remaining algebraic, numerical and probabilistic obligations . . . 36
6.5 The KG–NTT–MUL stopping boundary of frozen Week 16 KeyGen . . . 37
6.6 Post-freeze KG progress and the new semantic boundary . . . 38
6.7 Top-level functional correctness is still a specification . . . 39
6.8 The security composition formula is still a conditional interface . . . 39
6.9 Trust boundary and out-of-scope items . . . 39
6.10 Current claim matrix . . . 40

7 Dependency-ordered next steps and contribution points 41
7.1 The current single priority: repairing security ExpandVecA semantics . . . 41
7.2 Stage 1 complete: encoder . . . 42
7.3 Stage 2 complete: decoder . . . 43
7.4 Stage 3 complete: actual harness inverse . . . 43
7.5 Stage 4 complete: production full-HBZ wrapper . . . 44
7.6 Stage 5 complete: fixed all-6 success postcondition . . . 44
7.7 Stage 6 frozen stop: KG–NTT–MUL audit . . . 44
7.8 Stage 7 frozen: accepted Sign core . . . 45
7.9 Stage 8 frozen stop: actual Verify helper chain . . . 45
7.10 Stage 9 in progress: post-freeze actual avec/qj audit . . . 45
7.11 Treat the frozen product/replay line separately . . . 46
7.12 Problems to which pure algebraists can contribute directly . . . 46
7.13 Where algebraic-geometric skills are useful . . . 47
7.14 Checklist for submitting a new theorem . . . 47

A Theorem and source index 48
A.1 Status and scope documents . . . 48
A.2 Packed keys and memory bridges . . . 49
A.3 Frozen Week 16 MINCORE KeyGen, Sign and Verify boundaries . . . 50
A.4 Post-freeze shared NTT/Rq and actual avec/qj . . . 51
A.5 μ, challenge, actual API trace . . . 52
A.6 Signature prefix and HBZ leaf . . . 53
A.7 rANS tables, pure traces and actual boundaries . . . 54
A.8 Top-level goals and assumption surface . . . 57

B Glossary of symbols and terminology 58

C Reproduction and reading paths 59
C.1 Building only this guide . . . 59

C.2 The full proof pipeline 60
C.3 A 30-minute reading path 60
C.4 A half-day path before contributing proofs 61

1 Reading contract and overall map

1.1 What this guide assumes and explains

This guide assumes familiarity with rings, quotient rings, free modules, equivalence relations and quotient sets, commutative diagrams, and induction. It uses standard English mathematical terminology in introductions, headings, figures, and the glossary, with consistent terms in repeated contexts. It does not assume knowledge of the following.

- The design of HAETAE or module-lattice signatures;
- Jasmin syntax and memory models;
- EasyCrypt's Hoare logic, pRHL, or tactic language;
- Implementation conventions for rANS entropy coding.

The goal is not a line-by-line explanation of every source file. For each proof obligation, the guide asks: "What is the object?", "Which map must commute?", "Which premises restrict its domain?", and "Which arrows have currently been proved for actual procedures?" The project's scope is limited to mode 2 and pinned specification/implementation snapshots [13].

Translation principle 1.1 (Translation across three layers)**.** Read the repository's theorems as propositions moving between three layers.

1. **Algebraic layer**: R_q-module elements, decompositions, challenges, and paper algorithms;
2. **Representation layer**: canonical byte strings, prefix projections, memory regions, and fixed-width integers;
3. **Execution layer**: actual generated procedures, branches, loops, return values, and trace observations.

Transporting an equality from one layer to another always requires a named representation/refinement theorem. A shared mathematical intention does not make a diagram commute automatically.

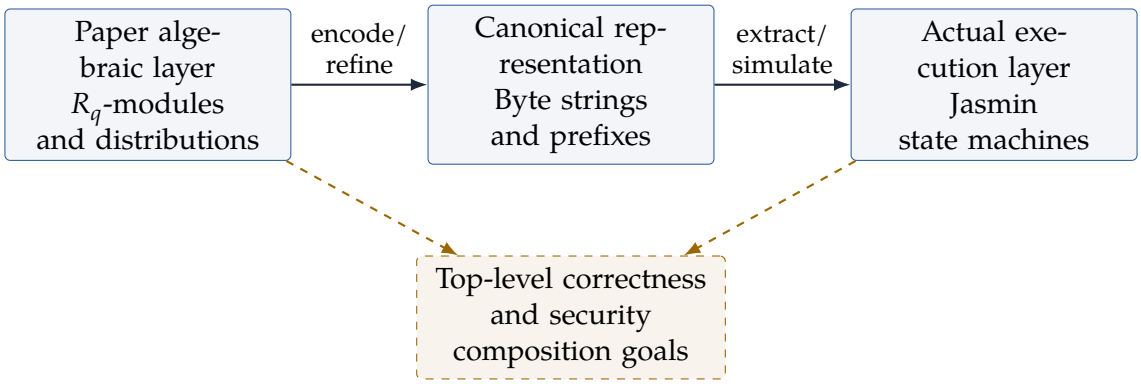


Figure 1: Overall proof structure. Only some horizontal arrows are closed; the top-level composition arrows remain partial.

1.2 Status terms include logical quantifiers

The operational source of truth is `CLAIM_LEDGER.md` [3]; dependencies are recorded in `THEOREM_GRAPH.md` [14]. This guide uses status terms only with the following meanings.

Status	Exact meaning	What does not follow automatically
PROVED	A named EasyCrypt declaration freshly compiles under the stated premises.	Parent claims, termination, losslessness, distributional equality
PARTIAL	Child propositions or restricted directions are proved, but the named parent remains open.	A global statement with the restrictions removed
SPECIFIED	Operations, predicates, or interfaces expressing the goal compile.	A proof of the goal
BLOCKED	A specific extraction, semantic, numerical, or probabilistic theorem is missing, preventing composition.	A claim that the obstacle is insurmountable
DEFERRED	Explicitly frozen at a paper boundary after a time limit.	A claim that the obligation is false or unnecessary

In particular, "Hoare partial correctness is proved" means

if the procedure returns, the postcondition holds

rather than that the procedure returns on every input or that a probabilistic retry terminates almost surely. This distinction is essential because of KeyGen and Sign rejection loops.

1.3 How far algebraic geometry applies

The ring R_q of course defines the affine scheme $\operatorname{Spec} R_q$. Local memory restrictions, local equalities, and frame conditions also suggest restriction maps and gluing. The current EasyCrypt files, however, do not formalize schemes, varieties, spectra, sheaves, or cohomology. Proofs chiefly concern arrays over finite sets, fixed-width words, state transitions, and integer division.

> **Analogy (not a formalized theorem).**
>
> Viewing memory m as a section over an address set, and equality on a region U as $m_1{\restriction_U} = m_2{\restriction_U}$, quickly suggests the shape of region-local hash theorems. The analogy does not use sheaf axioms or a scheme structure to derive EasyCrypt conclusions. The actual basis is pointwise `load8` equality and the procedure's read footprint.

Algebraic geometers should pay particular attention to hashes. SHAKE state transitions are generally neither R_q-linear maps nor regular morphisms. This guide treats hashing as a deterministic state machine reading finite byte strings. Equality of hash inputs therefore yields equality of outputs through relational execution theorems about the two actual programs, rather than through algebraic functionality.

1.4 Six statements to remember first

1. Top-level `FC_KeyGen`, `FC_Sign`, and `FC_Verify` are still goal interfaces, not completed theorems.

2. $\mathrm{prefix}_{992}(sk) = vk$ and raw/internal μ-prefix agreement are proved down to actual generated procedures, but do not automatically establish complete API security.

3. The pure HBZ/rANS inverse structure, individual actual encoder/decoder trace refinements, core-harness inverse including actual copying, and production full-HBZ wrapper arrows are closed as success-conditioned partial correctness.

4. For the all-6 stream generated by canonical all-zero HBZ, failure of a returning actual encoder is excluded and the production zero round trip is proved; fixed-input termination, losslessness, and `phoare` remain open.

5. The actual KeyGen matrix/finalize snapshot has a proved KG-2-like decomposition and snapshot-only mod-$2q$ congruence. At the frozen-paper stage, however, full-NTT convolution and the representation adapter following the final `output_row` NTT-word rewrite were absent, so work was frozen under `STOP-KG-NTT`. Sign was subsequently frozen under `STOP-SIGN-CHAL-MODE2`; Verify was frozen at `STOP-VERIFY-MATRIX-CRT`, lacking the actual 2-by-4 NTT/CRT/freeze procedure [24, 25]. Post-freeze work closed pure convolution, the native R_q row product, and R_q–HAETAE representation transport, and interpreted actual `avec` as the paper's a. But security-side synthetic ExpandVecA differs from actual SHAKE128/rejection semantics and the paper's qj, so the current status is `STOP-KG-AVEC-QJ`. This does not change the frozen 82-target verdict [12, 10, 4].

6. Separate final cryptographic assumptions from implementation proof obligations. Implementation correctness beyond the MLWE/BST-MSIS/ROM interfaces is not hidden as an assumption.

These six statements govern status judgments throughout the following chapters.

2 Algebraic objects: from quotient rings to signature bytes

2.1 The base rings and mode 2 parameters

In the pinned specification,

$$R = \mathbb{Z}[X]/(X^{256}+1), \qquad R_q = \mathbb{Z}_q[X]/(X^{256}+1), \qquad q = 64513$$

and the mode 2 module dimensions are $(k, \ell) = (2, 4)$. The small-secret-coefficient bound is $\eta = 1$, and the binary challenge has Hamming weight $\tau = 58$ [1]. Each R_q element is an equivalence class of polynomials of degree < 256; choosing coefficient vectors identifies it with $\mathbb{Z}_q^{256}$ as a set.

The specification chooses $q \equiv 1 \pmod{512}$, so $X^{256}+1$ splits completely over $\mathbb{F}_q$ and an NTT is available. On paper, one can therefore consider the Chinese remainder theorem isomorphism

$$R_q \simeq \prod_{i=1}^{256} \mathbb{F}_q$$

The repository's top-level implementation-security interface nevertheless retains `obl_public_key_algebra_ntt` as a separate obligation. Existence of this isomorphism and correctness of actual NTT routines must not be treated as the same claim.

Caution 2.1 (Separate mathematical background from implementation theorems)**.** The ring/module description in this section explains the pinned HAETAE specification. The implementation theorem that "actual Jasmin NTT computes this isomorphism correctly" requires a separate proposition and premises. This guide applies **PROVED** only to named theorems under `easycrypt/`.

2.2 Viewing KeyGen as a module map

In the paper's KeyGen data flow, a seed generates $a_{\text{gen}} \in R_q^k$ and $A_{\text{gen}} \in R_q^{k\times(\ell-1)}$. Small elements $s_{\text{gen}} \in R^{\ell-1}$ and $e_{\text{gen}} \in R^k$ are sampled, and

$$b = a_{\text{gen}} + A_{\text{gen}} s_{\text{gen}} + e_{\text{gen}} \in R_q^k$$

is computed. The coefficientwise decomposition

$$b = b_0 + 2b_1, \qquad (b_0, b_1) = (\text{LowBits}_{vk}(b), \text{HighBits}_{vk}(b))$$

then compresses the verification key to $vk = (seedA, b_1)$. The packed secret-key representation includes those bytes as a prefix so that Sign can read the same verification-key components.

The first composition seam closed in this repository is this inclusion relation, not the complete KeyGen distribution. Let S be the set of secret-key byte arrays and V the set of verification-key arrays, and define

$$\pi_{992} : S \longrightarrow \mathsf{Byte}^{992}, \qquad \pi_{992}(s) = s[0..992)$$

For a returning execution of actual mode 2 internal KeyGen with result (vk, sk),

$$\pi_{992}(sk) = \pi_{992}(vk)$$

The array on the right is larger; only its first 992 bytes are compared.

This can also be expressed precisely as a quotient. Define on S

$$s \sim_{992} s' \quad \Longleftrightarrow \quad s[0..992) = s'[0..992)$$

Then π_{992} classifies $S/\!\sim_{992}$. The current theorem states that the $\sim_{992}$-equivalence class of the packed secret key agrees with the public-key representation. It does not state equality of the entire secret keys.

2.3 The Week 16 KeyGen snapshot congruence

Week 16's transparent harness `ActualM23MatrixFinalizeSnapshot.run` calls actual `_kp_m23_matrix` with mode 2 dimensions $(2, 3)$, stores its returned bp as pre_bp, and calls actual `_keypair_finalize_m23` on 512 active words [23, 22, 21]. At coefficient i, let

$$r_i = (pre_bp_i + s_{2,i} + a_i) \bmod q$$

The compiled semantic snapshot gives

$$r_i = 2b_{1,i} + b_{0,i}, \qquad -1 \le b_{0,i} \le 1, \qquad s'_{2,i} = s_{2,i} - b_{0,i}$$

Moreover,

$$2(a_i - 2b_{1,i}) + 2pre_bp_i + 2s'_{2,i} \equiv 0 \pmod{2q}$$

holds at every active coefficient. Here pre_bp is the return snapshot of the actual matrix helper; it has not yet been identified with the paper's $A_{\text{gen}} s_{\text{gen}}$.

Caution 2.2 (Separate the snapshot congruence from the paper's KeyGen equation)**.** actual_m23_matrix_finalize_semantic_snapshot combines two actual calls, low/high decomposition, adjusted s_2, and the frame in one Hoare postcondition. output_row_from_mode2_ntt_words identifies `KeygenM23ArithmeticSpec.output_row` exactly with the actual full-invNTT/pointwise word representation. The frozen paper ended at `STOP-KG-NTT`, lacking the odd-root orthogonality/full-NTT convolution theorem

and representation adapter needed to transport that representation to native R_q negacyclic multiplication.

Post-freeze, rq_mul_coeff_foldr_to_bigi, full_ntt_montgomery_spectral_action, full_ntt_montgomery_row_product, keygen_mode2_row_product, and mode2_row_product_haetae_dot close this pure algebra and representation transport. The two actual `avec` rows are also interpreted as the paper's a, and qj from the paper's $j = (1, 0)^T$ is added as a separate term in the snapshot congruence. The existing security-side `haetae_mode23_qj_vector`, however, is a synthetic generator starting at coefficient 401, different from the paper's $qj[0, 0] = 64513$ and actual SHAKE128/rejection expansion. The congruence above must therefore not simply be renamed to the existing security object's $As = qj \pmod{2q}$, nor read as proving security-model KG-1–KG-4. The current boundary is `STOP-KG-AVEC-QJ`; the frozen-paper boundary remains unchanged [12, 10].

The latest standalone `KgFaithfulAugmentedModel.ec` does not conceal this gap by bypassing it: it builds a separate faithful coefficient carrier and instantiation with the actual matrix/secret. actual_faithful_augmented_productE expands the structured coefficient product on an active row as the sum of a head, generated HAETAE `poly_dot`, and adjusted error. checked_mode2_parent_m23_finalize_faithful_augmented_paper_as_qj combines the actual sampler's paper a, actual 3-term R_q row product, and adjusted error into one expression, proving at every active coefficient that `faithful_augmented_matrix_vector_product_coeff` is congruent to `faithful_qj_coeff` mod-$2q$. This is **PROVED**, with verdict `GO-KG-FAITHFUL-AUGMENTED`. The product evaluator accepts only the actual width-6 matrix/secret and consumes the head, middle three columns, and two identity columns. Its mod-$2q$ block-specific coefficient semantics avoid eliminating (qj) in a mod-q ring. It is therefore neither the existing width-4 security `matrix_vec_mul` theorem nor equivalence to synthetic security ExpandVecA.

2.4 The transcript where Sign and Verify meet

In the pinned specification, the first observation Sign and Verify must reconstruct in common is the message digest μ. Restricting attention to the relevant portion,

$$\mu = H_{\text{gen}}(seedA, b_1, M),$$

and the challenge input has the form

$$\rho = H(\text{HighBits}_h(w), \text{LSB}(\lfloor y_0 \rceil), \mu)$$

[1]. Sign reads $(seedA, b_1)$ from the packed secret key's 992-byte prefix; Verify reads the same bytes from public-key memory.

The central equality currently proved at the raw/internal generated boundary is

$$\underset{32}{\text{take}}(\mu^{64}_{\text{Sign}}) = \mu^{32}_{\text{Verify}}$$

Absorbing these 32 bytes from the same challenge state at position 64 then produces equal final states and positions. This is a relational theorem directly including the actual generated hash/absorb procedures.

The entire ρ is not yet proved. In particular, actual Sign–Verify agreement of $\text{HighBits}_h(w)$ and the LSB representation, the complete challenge call, and distribution/rejection analysis remain separate obligations. The closed μ square therefore cannot be extended to the complete challenge or to full $\delta_{\text{Sign}} = 0$.

2.5 Signature representation is a partial isomorphism on a canonical subset

A mode 2 signature has 1474 bytes, with the actual extracted layout

$$\underbrace{[0,32)}_{\text{256 challenge bits}} \;\|\; \underbrace{[32,1056)}_{\text{1024 signed low bytes}} \;\|\; \underbrace{[1056,1058)}_{\text{two size deltas}} \;\|\; \underbrace{[1058,1474)}_{\text{HBZ/h payload and padding}}$$

Let C_{prefix} be the set of canonical inputs in which every bit word is 0 or 1 and every low word lies in the signed-byte range. For the actual prefix packer and unpacker,

$$\operatorname*{Dec}_{\text{prefix}} \circ \operatorname*{Enc}_{\text{prefix}} = \mathrm{id}_{C_{\text{prefix}}}$$

is currently proved as partial correctness.

The domain restriction is substantive. On noncanonical words, encoding may discard information or decoding may return a different representative. In algebraic terms, C_{prefix} is a chosen system of representatives, and the round trip is a section–retraction relation on that system.

This equality is not yet available for the entire signature space. Size metadata, HBZ and h payloads, canonical zero padding, and success and uniqueness of the complete parser remain open.

2.6 Which algebraic facts belong to which layers

Object	Required proposition	Current status
R_q-module KeyGen	Equality between Jasmin operations and the paper's KeyGen distribution	**PARTIAL**/ top-level goal
actual KeyGen snapshot	Low/high decomposition of matrix/finalize results and snapshot-only mod-$2q$ congruence	**PARTIAL** (KG-2-like boundary proved; NTT product bridge blocked)
packed key	$\pi_{992}(sk) = \pi_{992}(vk)$	**PROVED** (partial correctness)
raw μ transcript	Equality of the first 32 bytes of Sign's 64-byte output and Verify's 32 bytes	**PROVED** (generated raw/internal)
challenge suffix	Equal states after absorbing μ_{32} from the same position 64	**PROVED** (local zero-loss)
signature prefix	Decode-after-encode is the identity on canonical inputs	**PROVED** (prefix only)
HBZ/rANS suffix	Inverse relation of the actual full encoder and decoder	**PROVED** (production wrapper and fixed all-6 Hoare success; termination unproved)
complete signature scheme	KeyGen/Sign/Verify refinement and security composition	**SPECIFIED**/ **BLOCKED**

The next section explains the logical distinctions between "partial correctness", "local", "generated", and "actual" in this table.

3 A dictionary for representation maps and program logic

3.1 Arrays and memory are different objects

EasyCrypt extractions contain both fixed-length arrays and global memory. Conceptually,

$$\mathsf{Byte} = \{0, 1, \ldots, 255\}, \qquad \mathsf{BArray}_N \simeq \mathsf{Byte}^N$$

and global memory can be read as a function on 64-bit addresses:

$$\mathsf{Mem} \simeq (\mathbb{Z}/2^{64}\mathbb{Z}) \longrightarrow \mathsf{Byte}.$$

The actual model has load/store operations and word representations, giving it more structure than a set-theoretic function, but this viewpoint suffices for reading interval equalities.

The relation saying that local array element i equals the ith byte after a memory base address has the form

$$\forall, 0 \le i < n, \qquad a_i = \mathsf{Mem}(base + i)$$

`Mode2KeyMemoryBridge.ec` and `ApiKeyMemoryBridge.ec` connect this relation to actual importer/exporter procedures. This is why an array equality alone cannot justify a conclusion about public pointers.

Definition 3.1 (Interval restriction and region equality)**.** For an address region $U = [b, b + n)$, let $m\restriction_U$ be its byte function. Read the local relation between two memories as

$$\mathrm{RegionEq}(m_1, m_2; b_1, b_2, n) \iff \forall, 0 \le i < n, m_1(b_1 + i) = m_2(b_2 + i)$$

Neither the base addresses nor memory outside those regions must be equal.

Week 6's region-local μ theorem requires only this relation on key, pre, and message regions, rather than whole-memory equality. It is the minimal relation matched to the data actually read by the hash procedure.

3.2 Two address representations and the canonical range

The raw ABI contains both integer addresses and $W64$ word addresses. The map

$$\mathbb{Z} \xrightarrow{\text{of_int}} \mathbb{Z}/2^{64}\mathbb{Z} \xrightarrow{\text{to_uint}} [0, 2^{64}) \cap \mathbb{Z}$$

is not the identity on all integers. Only in the canonical range excluding wraparound do we have $\text{to_uint}(\text{of_int}(a)) = a$.

Thus **PROVED** for **OBL-API-ADDRESS-BINDING** denotes a restricted bridge showing that the two address representations identify the same bytes, not complete memory safety. Valid-region and no-wrap premises describe the theorem's domain; they are not dispensable implementation details.

3.3 A procedure is a partial state transition, not a total function

One might wish to write a deterministic procedure f as a map from a state space X to Y. With retries or possible infinite loops, however, the natural object is a partial function

$$f : X \rightharpoonup Y$$

or a relation containing only terminating executions. This guide reads the Hoare proposition

$$\{P\}\, f\, \{Q\}$$

as

$$\forall x \in X, \quad P(x) \wedge f(x)\downarrow \Longrightarrow Q(x, f(x)).$$

Here $f(x)\downarrow$ means that execution returns. The proposition alone does not prove $f(x)\downarrow$.

Translation principle 3.2 (Hoare theorem)**.** Read `hoare[P:pre==>post]` as saying that, on the subspace defined by pre, the graph of the actual procedure lies in the post subset whenever execution terminates. KeyGen prefix theorems and codec round trips have this form.

A relational theorem for two programs f_1, f_2 takes the form

$$\mathrm{equiv}[f_1 \sim f_2 : P \Longrightarrow Q]$$

This is a pRHL proposition relating final states of two terminating executions by Q when their initial states are related by P. Variables on the left and right use EasyCrypt notation $x\{1\}$ and $x\{2\}$.

Translation principle 3.3 (Relational theorem)**.** A pRHL theorem can often be read as a simulation between two state machines or a fragment of observational bisimulation. If the current file proves only a one-directional relation on results, however, it is not described as a full bisimulation.

3.4 Refinement is a commutative square

Given an abstract operation F, an actual procedure f, and input/output representation maps e_X, e_Y, a typical goal is

$$\begin{array}{ccc} X & \xrightarrow{F} & Y \\ \downarrow e_X & & \downarrow e_Y \\ X_{\mathrm{bytes}} & \xrightarrow{f} & Y_{\mathrm{bytes}} \end{array} \qquad e_Y \circ F = f \circ e_X$$

Rather than close this entire square in one step, the repository breaks it down as follows.

1. A pure array invariant or integer identity;
2. A Hoare theorem for an extracted leaf procedure;
3. An exact adapter showing that the generated wrapper yields the same result as the leaf;
4. Raw-API importer/exporter procedures and address bridges;
5. Actual caller branches and memory frames;
6. Top-level functional, probabilistic, and security composition.

Each stage uses a different state space. Composition is possible only after checking that the lower-stage theorem actually establishes the upper stage's premises.

3.5 A frame is a theorem about write support

If a procedure writes only memory region W, its frame property typically has the form

$$a \notin W \Longrightarrow m_{\text{after}}(a) = m_{\text{before}}(a)$$

or, under the premise that the region of interest U is disjoint from W,

$$m_{\text{after}}{\restriction}_U = m_{\text{before}}{\restriction}_U$$

Week 6's actual Sign output frame shows that the 1474-byte signature write and 8-byte signature-length write preserve disjoint VK/SK/pre/message regions. Without this property, one cannot justify saying that Verify reads the same key/message after Sign.

> **Analogy (not a formalized theorem).**
>
> The frame rule resembles a proposition that a function's support lies in W. If the object of interest is restricted to U and $U \cap W = \emptyset$, the change is invisible under restriction. The actual proof uses pointwise byte-load/store equalities rather than support or sheaf terminology.

3.6 Canonicality chooses representatives

Three types of premise recur in codec theorems.

- Each challenge word is exactly 0 or 1;
- Each signed low coefficient lies in the specified byte range;
- Each HBZ coefficient lies in $[-6, 7)$.

These representative conditions make decoding a left inverse of encoding. In general, encoding maps a larger word space into a byte space and may discard information, so it need not be injective on its entire domain. The precise goal is to prove, on a canonical subset C,

$$\text{Dec} \circ \text{Enc}{\restriction}_C = \text{id}_C$$

where the right-hand side is the identity map. Preservation of the unused output-array tail, unique padding, and parser nonfailure are separate frame and canonical-parsing theorems.

3.7 Exact traces and ghost observations

Actual raw callers perform substantial computation before and after hash calls. For analysis, trace modules preserve actual results and final memory while recording intermediate *observed_mu*, branch reachability *tail_reached*, and actual hash-descriptor inputs in ghost fields.

An exact-trace theorem establishes roughly

$$\text{obs}(\text{actual execution}) = \text{obs}(\text{instrumented trace})$$

including actual return values and memory. Defining a ghost field, however, does not automatically establish a mathematical relation on its value. In particular, equality of *observed_mu* in two distinct completed traces requires separate product/replay reasoning to transport a return-value pRHL theorem into trace post-states.

3.8 The exact scope of "zero-loss"

A pRHL theorem showing that two deterministic generated procedures produce equal observations on related inputs permits statistical loss 0 for that *local seam*. This is why the current result can be summarized as

$$\delta_{\mu,\text{raw_top}} = 0$$

The following extension is not valid:

$$\delta_{\mu,\text{raw_top}} = 0 \not\Rightarrow \delta_{\text{Sign}} = \delta_{\text{Verify}} = \delta_{\text{Encoding}} = 0.$$

The larger δ terms include sampler distributions, rejection/termination, the complete codec, actual API composition, and the remaining challenge inputs.

3.9 Reading one EasyCrypt theorem as a mathematical statement

In `ExactMuTopControl.ec`, sign_verify_generated_raw_mu_prefix directly compares the actual generated procedures `_sf_mu_rawpre` and `__verify_hash_mu`. Its premises can be grouped mathematically as follows.

1. The two executions satisfy the same global-memory relation and use the same pre/message descriptors.
2. The first 992 bytes of Sign's local secret-key array equal Verify's key region in memory.
3. Address regions are canonical and do not wrap.
4. Verify's prelength selects the actual raw branch.

The conclusion is

$$\operatorname{take}_{32}(\text{return}_{\text{Sign}}) = \text{return}_{\text{Verify}}$$

Omitting any of "actual generated", "return value", "raw branch", or "32-byte prefix" would overstate the theorem.

4 The proof chain currently completed

"Verified" in this section refers to the 82 authored targets in the 13 August 2026 snapshot. The count indicates project size, not completion of every parent claim below. Exact premises and EasyCrypt names for each proposition appear in appendix A.

4.1 Three main lines of progress

The current work is easiest to understand as three connected lines with different statuses.

1. **Key/transcript line**: from the packed-key prefix down to raw μ and the challenge suffix, then up to actual API memory/traces;
2. **Signature-codec line**: closing canonical signature-prefix and HBZ/rANS suffix inverses down to actual pack/unpack procedures;
3. **MINCORE KeyGen line**: transporting the algebraic result of the actual matrix/finalize snapshot to the paper's R_q-module product.

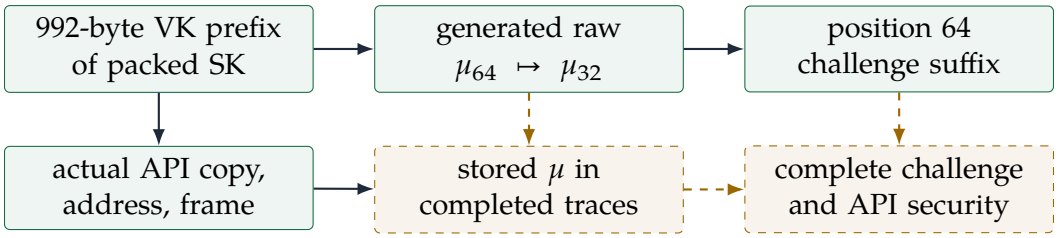


Figure 2: The key/transcript line. Solid arrows denote completed theorems; dashed arrows denote open lifts.

4.2 Stage 1: the verification-key quotient of a packed secret key

At the pure-array level, define

$$\mathrm{VKPrefix}_{992}(sk, vk) \iff \forall, 0 \le i < 992,\ sk_i = vk_i$$

`PackedKeyPrefix.ec` proves these basic facts.

- The byte-by-byte copy loop invariant is preserved at the next index;
- Copying all 992 bytes establishes the prefix relation;
- Writes from index 992 onward leave the completed prefix unchanged.

These array lemmas lift to the actual extracted procedure.

Verified proposition 4.1 (Prefix of the actual packer)**.** Under the mode 2 constants, pack_sk_m23_mode2_vk_prefix shows that when actual generated `_pack_sk_m23` returns, the first 992 bytes of its secret-key array equal the input verification-key array. The subsequent loops writing the eta-vector and key preserve this prefix.

Verified proposition 4.2 (Internal KeyGen return)**.** keypair_full_m23_mode2_return_prefix and keypair_internal_mode2_return_prefix lift this relation to actual generated KeyGen return values.

$$\text{KeyGen returns } (vk, sk) \implies \mathrm{VKPrefix}_{992}(sk, vk).$$

Both are **PROVED** partial-correctness theorems. They do not prove termination of the KeyGen rejection loop, its complete output distribution, or equivalence to the paper's KeyGen. Thus **FC-KG-TOP** remains **PARTIAL**.

4.3 Stage 2: from local arrays to public memory

Using the array relation returned by KeyGen as actual API input to Sign/Verify requires four kinds of arrow.

1. Exporters copying local KeyGen VK/SK arrays to external addresses;
2. The importer bringing external SK into Sign's local array;
3. The importer bringing external VK into Verify's local array;
4. A canonical bridge showing that integer and $W64$ addresses describe the same regions.

`ApiKeyMemoryBridge.ec` proves mode 2's 992/1408-byte prefixes and disjoint frames for the actual copy helpers.

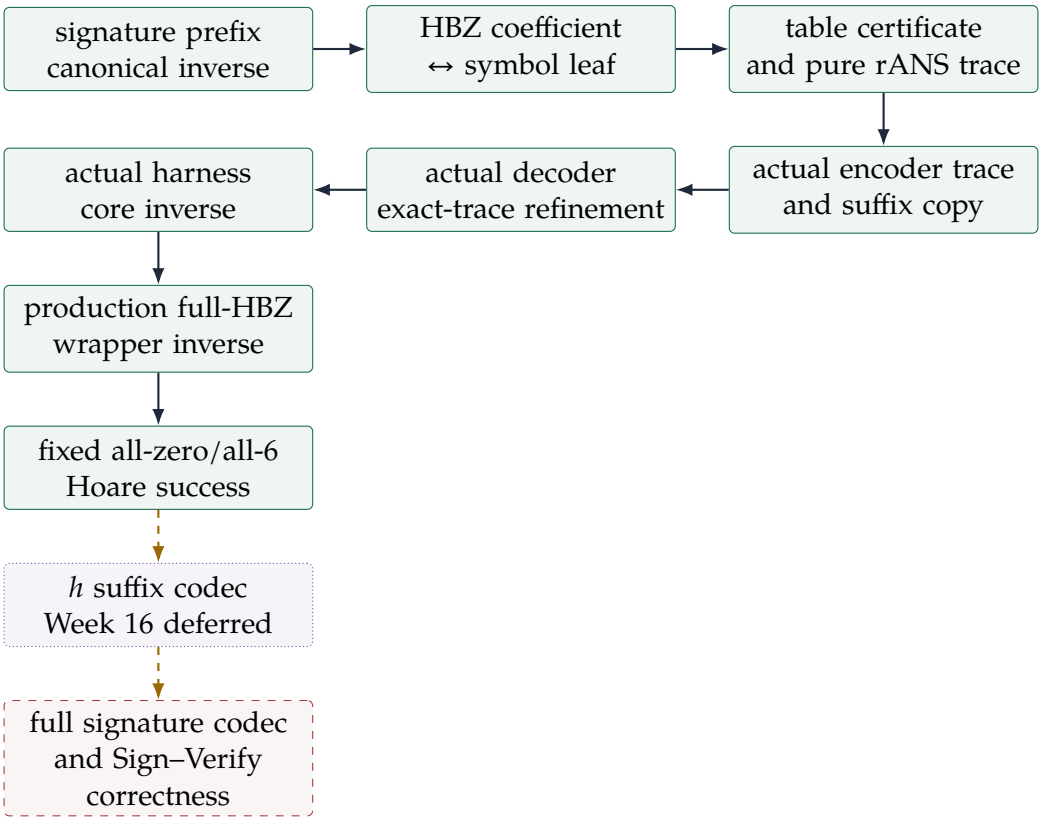


Figure 3: The signature-codec line. Week 11 closed the actual encoder's successful suffix, Week 12 the actual decoder consuming an exact trace, Week 13 core composition with actual copying in between, Week 14 the production full-HBZ wrapper boundary, and Week 15 fixed all-6 failure exclusion. The h codec was frozen when Week 16 switched to MINCORE.

Verified proposition 4.3 (Sequential exports of actual raw KeyGen)**.** keypair_raw_api_exports_matching_prefixes shows that when actual `cryptolab_haetae_mode2_keypair_internal` returns, the first 992 bytes of the external SK and VK regions agree under explicit valid/canonical/disjoint-region contracts.

This is stronger than merely saying that a copy function copies, but weaker than complete memory safety. Pointer validity and separation are explicit premises, and KeyGen termination is still not a conclusion.

4.4 Stage 3: generated raw μ for Sign and Verify

Sign absorbs the local packed-SK prefix and squeezes 64 bytes of μ. Verify absorbs the VK region in global memory and squeezes 32 bytes. The actual generated helper chain performs initialization, key/pre/message absorption, Keccak permutation, finalization, and squeezing through different procedure interfaces.

`ExtractedMuHashPrefix.ec` proves leaf relations, and `ExactMuTopControl.ec` lifts them to the actual top procedures.

Verified proposition 4.4 (Generated raw μ prefix)**.** Under explicit mode 2 key-prefix, equal pre/message-region, raw-branch, and valid/no-wrap premises,

$$\mathsf{SignRawMu} \sim \mathsf{VerifyHashMu}$$

holds with conclusion

$$\operatorname{take}_{32}(\mu^{64}_{\mathrm{Sign}}) = \mu^{32}_{\mathrm{Verify}}$$

The named theorem is sign_verify_generated_raw_mu_prefix.

This theorem places the two actual generated procedures directly on its theorem surface. It does not simply identify an abstract wrapper proposition with actual procedures. In addition, raw_prelen_shr63_zero and raw_prelen_branch_guard show that the actual bit-level branch selects the raw path.

4.5 Stage 4: from whole-memory equality to read-region equality

Once actual Sign has written a signature, there is no reason for the two executions' entire global memories to agree. Only the key, pre, and message regions read by the hash are needed.

Verified proposition 4.5 (Region-local generated hash)**.** sign_verify_generated_raw_mu_prefix_regionwise allows different global memories. When the key/pre/message read regions are related pointwise, actual generated Sign/Verify raw-hash return values share the same 32-byte prefix.

Verified proposition 4.6 (Sign output frame)**.** sign_raw_api_frames_reused_regions shows that actual raw Sign preserves VK/SK/pre/message regions disjoint from its outputs even while writing 1474 signature bytes and 8 siglen bytes.

Together, these theorems establish the memory-level basis that the input bytes Verify will read remain after Sign. The final transport directly comparing stored μ in two completed traces is still open, however. That obligation is separated in section 6.

4.6 Stage 5: the position 64 challenge suffix

Suppose Sign and Verify have the same challenge state and position, the position is 64, and the preceding stage gives equal μ_{32}. A theorem shows that running each actual generated absorb helper once leaves equal states and positions.

Verified proposition 4.7 (Local challenge-suffix zero-loss)**.** generated_raw_mu_to_challenge_suffix_zero_loss sequentially calls the actual generated raw μ procedure and actual generated 32-byte challenge absorb helper on each side. Under the stated premises, the final $(state, position)$ pairs agree.

Deterministic loss is therefore zero at the raw/internal generated seam. This theorem does not include the highbits/LSB prefix, complete challenge, stored μ in an actual accepted API trace, or the context branch.

4.7 Stage 6: actual caller control and accepted Verify

Weeks 4–6 placed these leaf results into actual public/raw caller contexts. The following currently compile separately.

- Exact μ trace and hash-input binding for actual raw Sign;
- An exact trace preserving every early-reject/tail branch of actual raw/cryptolab Verify;
- The implication from Verify acceptance to reaching the actual tail and hash call;
- Binding showing that accepted Verify used actual VK/pre/message descriptors;
- An exact trace sequentially executing actual Sign and then actual Verify;
- A frame preserving key/pre/message regions during that sequential execution.

Verified proposition 4.8 (From acceptance to the tail)**.** For actual raw/cryptolab Verify,

$$\mathit{VerifyResult} = 0 \Longrightarrow \mathit{tail_reached}$$

is proved. This is neither the reverse implication nor a statement that an arbitrary 1474-byte signature reaches the tail.

Verified proposition 4.9 (Actual sequential trace)**.** raw_sign_then_verify_actual_exact_trace matches return values and final memory of actual raw Sign followed by actual raw Verify to sequential execution of two instrumented traces.

Nevertheless, **OBL-DIRECT-OBSERVED-MU** is **PARTIAL**. An exact trace exposes observation locations and values, but does not automatically transport a separate return-value pRHL theorem into equality of ghost fields in two already completed traces.

4.8 Stage 7: section–retraction for the actual signature prefix

`Mode2SignaturePrefixPack.ec` and `Mode2SignaturePrefixUnpack.ec` expose the exact layout of the actual generated prefix loops. `Mode2SignaturePrefixRoundTrip.ec` contains the harness calling both procedures in their actual order.

Verified proposition 4.10 (Mode 2 prefix round trip)**.** For canonical challenge and canonical signed-low inputs, pack_unpack_sig_prefix_mode2_roundtrip establishes

$$\begin{aligned} \mathit{decodedChallenge}[0..256) &= \mathit{challenge}[0..256), \\ \mathit{decodedLow}[0..1024) &= \mathit{low}[0..1024) \end{aligned}$$

and frames the unused decoded-low tail.

An all-zero witness for the canonical premises also compiles, so the proposition is nonvacuous. The result nevertheless covers only the signature's first 1056 bytes. It excludes the two metadata bytes, 416-byte suffix, padding, and success of the complete unpacker.

4.9 Stage 8: the HBZ coefficient–symbol leaf

HBZ input comprises 1024 coefficients $h_i \in [-6, 7)$. Prepare computes $s_i = h_i + 6 \in \{0, \ldots, 12\}$, and apply restores $s_i - 6$ as a 32-bit word.

Verified proposition 4.11 (HBZ leaf inverse)**.** For a canonical HBZ array, encode_prepare_decode_apply_mode2_inverse shows that sequentially executing actual prepare and actual apply restores the first 1024 coefficients, preserves the decoder coefficient tail, and makes prepare's *bad* output zero.

This is a leaf-level inverse bypassing the rANS core. It alone does not establish a round trip for the actual compressed suffix.

4.10 Stage 9: actual tables and pure rANS mathematics

Mode 2 HBZ's 13 frequencies sum to 1024, and their cumulative intervals partition $[0, 1024)$. `Mode2HbzTableCertificate.ec` proves this arithmetic as well as the meaning of reciprocal/bias values and packed decoder words. Separate files compile theorems that the actual generated literal arrays satisfy that certificate.

Verified proposition 4.12 (Pure single-step inverse)**.** For each symbol s and admissible state x, using c_s, f_s from the actual table gives

$$D_s(E_s(x)) = x$$

The reciprocal-based fast encoder step also equals mathematical E_s in the stated normalized range.

Week 9 defined pure *xs*/*cuts*/*bytes* traces including normalization bytes between individual steps.

Verified proposition 4.13 (Pure byte stack)**.** For canonical symbol lists, the following compile.

- Normalization emits only 0, 1, or 2 bytes at each step;
- Appending emitted segments in decoder order restores the original state;
- The 4-byte little-endian initial state is recovered after parsing;
- Every trace state stays in $[2^{23}, 2^{31})$;
- The trace's head symbol, reduced state, and normalization segment decode one step at a time.

Verified proposition 4.14 (Actual copying and control harness)**.** copy_encoded_suffix_correct proves pointwise slice copying and the tail frame of actual `__copy_encoded_suffix`. actual_rans_harness_branches_on_encoder_result shows that the actual encoder, copy, and decoder are called in that order; the decoder runs exactly when the encoder's actual *bad* result is zero, receiving $(count, size, m) = (1024, size, 13)$.

The control theorem itself concludes neither decoder success nor symbol equality, final state, or byte-consumption count. Week 12's separate decoder theorem proves those semantics under the exact-trace input relation, but at that stage had not yet composed them into the control harness. Week 13's actual_rans_encode_copy_decode_inverse adds those semantic conclusions to the same harness.

4.11 Stage 10: local semantics of the actual encoder

Week 10 incorporated 6 rANS encoder targets into the authored target manifest and freshly compiled them. Rather than immediately assert one large theorem between pure traces and actual generated procedures, these separate array/list correspondence, word arithmetic, inner normalization, and final serialization [15, 7].

Verified proposition 4.15 (Array, word, and serialization lemmas)**.** encode_trace_suffix_extension transports the actual 1024-byte symbol-array suffix to a pure trace recurrence extending it one symbol at a time. actual_mode2_encoder_word_step_correct shows that the actual word expression using literal-table reciprocals, shifts, biases, and complements equals the pure fast step. actual_encoder_final_state_serialization shows that four `set8` operations equal little-endian serialization of the 32-bit state.

Verified proposition 4.16 (Actual inner loop and successful-output size)**.** actual_rans_encode_inner_no_underflow installs byte-segment and prefix-frame invariants in the nested normalization loop of actual generated `_rans_encode`, establishing no underflow of the $W64$ cursor. actual_rans_encode_success_size_bound is an actual partial-correctness theorem: if that procedure returns, then

$$bad \neq 0 \quad \vee \quad (bad = 0 \wedge 0 \leq off \leq 1020 \wedge 4 \leq 1024 - off \leq 1024)$$

At the end of Week 10, this was stronger than the earlier $bad \in \{0,1\}$ control theorem, but did not state that the entire suffix returned by the actual procedure equaled TraceBytes(SymbolList($symbols_0$)). The inner phase then used a conservative 0..4 range. Procedure-level composition of the actual outer iteration's one-symbol state transition and final four stores remained open.

4.12 Stage 11: global trace closure of the actual encoder

Week 11 added 7 encoder targets, composing Week 10's local transitions into the entire successful output [16, 7]. The central invariant remembers both the initial array enc_0 and the already processed pure suffix, rather than an arbitrary inner-loop entry array.

Verified proposition 4.17 (Tail-aware invariant and generated arithmetic)**.** The proposition `encoder_inner_tail_inv` states that the actual inner loop preserves

$$\text{InnerWrittenSuffix}(x_0, s, k) \mathbin{+\!\!+} \text{EncodeTrace}(tail).bytes$$

after the current cursor.

Lemma encoder_inner_tail_exit_exact recovers $k = \text{NormalizationLen}(x_0, s) \leq 2$ when the actual guard exits. Lemma generated_loaded_nested_word_update transports actual table loads, reciprocal high products, shifts, complements, and biases to the pure fast step.

Verified proposition 4.18 (Encoder trace-closure theorem)**.** actual_rans_encode_trace_closure directly opens actual generated `RansEncodeTarget.M._rans_encode`. If it returns, it either fails or, on success, establishes all of

$$bad' = 0, \qquad 0 \leq off' \leq 1020, \qquad 4 \leq 1024 - off' \leq 1024,$$
$$\text{SegmentMatches}(enc', off', \text{TraceBytes}(\text{SymbolList}(symbols_0))),$$
$$off' + |\text{TraceBytes}(\text{SymbolList}(symbols_0))| = 1024, \qquad \text{PrefixFrame}(enc_0, enc', off')$$

actual_rans_encode_trace_refinement is the corollary expanding the same conclusion on the parent refinement surface.

The theorem also composes the fact that the final four generated stores prepend the serialized state to the accumulated normalization suffix. Thus **OBL-RANS-ACTUAL-INNER-NORMALIZE**, **OBL-RANS-FINAL-SERIALIZATION**, and **OBL-RANS-ENCODE-REFINEMENT** are each **PROVED** within their stated scopes. This remains Hoare partial correctness: it gives neither termination of an actual call nor existence of an execution with $bad' = 0$. The theorem alone therefore cannot establish success for a fixed input. At Week 11, **OBL-RANS-ACTUAL-SUCCESS-WITNESS** was **PARTIAL**; Week 15's separate all-6 theorem excludes failure in returning fixed-input executions.

4.13 Stage 12: exact-trace semantics of the actual decoder

Week 12 added 8 decoder targets, installing pure-trace coordinates into the actual generated decoder's two nested loops [17, 6]. The central input predicate `actual_mode2_decoder_trace_input` does not accept an arbitrary buffer; it jointly requires the following.

- The expected symbol array is a canonical mode 2 symbol stream;
- $encoded_size = |\text{TraceBytes}(\text{SymbolList}(expected))|$ and $4 \leq encoded_size \leq 1024$;
- $buffer_0[0..encoded_size)$ agrees pointwise with that exact trace;

- Each generated normalization read obtains the corresponding byte of the pure trace segment;
- Actual state fields are $(count, size, m) = (1024, encoded_size, 13)$;
- The actual mode 2 `symbol_words` and `dsyms_words` are used.

Verified proposition 4.19 (Decoder trace coordinates and invariant)**.** `decoder_state_at`, `decoder_cursor`, and `decoder_segment_at` turn the pure *xs*/*cuts*/*bytes* trace's state, cursor, and normalization segment into per-iteration coordinates. The proposition `decoder_outer_trace_live` jointly preserves

$$x = \mathrm{DecoderStateAt}(i), \qquad off = \mathrm{DecoderCursor}(i)$$

with equality of the decoded prefix and an untouched-tail frame. The inner invariant replays exactly 0/1/2 bytes of the corresponding segment, restoring the next pure state.

Verified proposition 4.20 (Actual decoder exact-trace theorem)**.** actual_rans_decode_trace_refinement directly opens actual generated `RansDecodeTarget.M._rans_decode` and shows, if it returns,

$$bad' = 0, \qquad off' = encoded_size,$$
$$\forall i,\ 0 \le i < 1024 \Longrightarrow decoded'_i = expected_i,$$
$$\forall i,\ 1024 \le i < 2048 \Longrightarrow decoded'_i = decoded_{0,i}$$

It also derives the internal exit state $x = 2^{23}$, passing the actual final reject test. Because the generated ABI does not return x, however, only returned arrays and state fields are exposed in the postcondition.

Thus **OBL-RANS-DEC-NORMALIZE** and **OBL-RANS-DECODE-REFINEMENT** are **PROVED** as exact-trace Hoare partial correctness. At the end of Week 12, no sequential-harness theorem transported the encoder's `segment_matches` and actual copying's `slice_eq` into this input predicate. The next Week 13 stage closes precisely that edge.

4.14 Stage 13: the actual encoder–copy–decoder core inverse

Week 13 added 2 targets and composed the three calls of actual `Mode2RansActualHarness.run` in order [18, 5]. The harness and control theorem are in `Mode2RansActualInverse.ec`; semantic closure is separated into `Mode2RansCoreCompositionBridge.ec` and `Mode2RansCoreActualInverse.ec`. The new bridge provides these transport lemmas.

- copied_suffix_is_exact_trace: obtains an exact trace at offset 0 of the copied buffer from encoder `segment_matches` and actual-copy `slice_eq`;
- trace_bytes_trace_segment_nth: decomposes global `trace_bytes` coordinates into per-symbol local-segment and decoder-cursor coordinates;
- segment_matches_implies_exact_decoder_segment_input: obtains every pointwise decoder-byte-read premise from the global trace segment; segment_matches_implies_decoder_word_reads transports these into actual W64 reads;
- actual_decoder_input_from_copied_trace and configured_decoder_state_fields: turn the copied trace and the three actual state stores, respectively, into components of the decoder input relation;
- actual_decoder_input_from_configured_trace: constructs the entire Week 12 decoder input predicate from the copied trace, canonical symbol stream, size bound, and three actual state stores;

- encoder_success_size_word_bridge: identifies actual (W64) subtraction with integer trace size in the no-wrap range.

Verified proposition 4.21 (**OBL-RANS-CORE-INVERSE**)**.** The semantic preconditions of actual_rans_encode_copy_decode_inverse are only harness-argument binding and `mode2_hbz_symbol_stream`. It does not assume encoder success, copied-trace equality, decoder execution, or decoder success. If it returns, one of two branches holds:

$$\begin{aligned}
\text{failure}: \quad & encoderBad \neq 0, \quad \neg decoderRan, \quad decoderOff = 0, \quad decoderBad = 1, \\
& decoded' = decoded_0, \quad decoderState' = decoderState_0; \\
\text{success}: \quad & encoderBad = 0, \quad decoderRan, \quad (count, size, m) = (1024, encodedSize, 13), \\
& decoderBad = 0, \quad decoderOff = encodedSize, \\
& decoded'[0..1024) = expected[0..1024), \\
& decoded'[1024..2048) = decoded_0[1024..2048).
\end{aligned}$$

The success branch also preserves the actual encoder-suffix relation and initial prefix frame. The core inverse of the three actual generated procedures therefore compiles as success-conditioned partial correctness.

core_composition_preconditions_satisfiable uses an all-zero canonical symbol array to show only nonvacuity of the premises. Because the conclusion is a failure/success disjunction, neither reachability of the actual success branch nor encoder/decoder termination follows. Thus **OBL-RANS-ACTUAL-SUCCESS-WITNESS** remained **PARTIAL** at Week 13. This conclusion did not automatically lift to production full-HBZ wrappers; the separate lift was closed in Week 14 and fixed all-6 failure exclusion in Week 15.

4.15 Stage 14: the production full-HBZ wrapper boundary

Week 14 added 5 targets connecting focused full-HBZ procedures to the actual SignaturePack/Unpack extraction boundary [19, 9]. The proof chain has four layers.

1. `Mode2HbzInternalBoundaries.ec` exactly connects prepare, rANS encode/decode, and apply calls inside the focused full wrappers to existing actual-procedure theorems.
2. actual_encode_hb_z1_full_mode2_trace preserves the failure/success branches of the actual full encoder on canonical HBZ coefficients. On success it provides a prepared-symbol witness, exact trace size/segment, and suffix frame.
3. actual_decode_hb_z1_full_mode2_inverse shows that, given that exact trace and prepared-symbol relation, the actual full decoder returns $bad = 0$, the original 1024-coefficient prefix, and the coefficient-tail frame.
4. actual_hbz_full_encode_decode_inverse_mode2 composes the focused encode/decode harness, and signature_pack_unpack_hbz_full_actual_exact matches it exactly to the production SignaturePack/Unpack harness, including the result tuple and `decoder_ran`.

Verified proposition 4.22 (**OBL-SIG-HBZ-ENCODE-DECODE**)**.** The only semantic preconditions of the Hoare corollary signature_pack_unpack_hbz_full_inverse_mode2 are argument binding and `canonical_hbz_mode2`. If execution returns, this disjunction holds:

$$\begin{aligned}
\text{failure}: \quad & size = 0, \quad \neg decoderRan, \quad encoded = out_0, \\
& decoded = decoded_0, \quad bad = bad_0; \\
\text{success}: \quad & size \neq 0, \quad 4 \leq size \leq 1024, \quad decoderRan, \quad bad = 0, \\
& decoded[0..1024) = hbz_0[0..1024), \\
& \text{CoeffTailFrame}(decoded_0, decoded), \quad \text{SuffixFrame}(out_0, encoded),
\end{aligned}$$

and the success branch has a prepared-symbol witness with `segment_matches` for its `mode2_trace_bytes`. Production-wrapper composition is therefore **PROVED** as success-conditioned partial correctness.

The theorem does not assume encoder success; it preserves the actual failure branch. The Week 14 theorem itself, however, proves neither failure exclusion for a returning execution on canonical all-zero HBZ entering the actual encoder's nonzero-size branch, nor encoder/decoder termination. The fact that prepare produces an all-6 symbol stream and the actual encoder's success are distinct propositions. The former is composed with Week 15's separate budget/failure-cause theorem; termination remains separate.

4.16 Stage 15: fixed all-6 actual-success postconditions

Week 15 added 2 targets, closing **OBL-RANS-ACTUAL-SUCCESS-WITNESS** as fixed-input Hoare partial correctness [20, 8]. The first target, `Mode2RansAllSixBudget.ec`, proves the following pure/prepare chain.

- zero_hbz_get32, zero_hbz_canonical, and actual_prepare_zero_hbz_all_six establish $bad = 0$ and a 1024-element all-6 prefix in the actual prepare return;
- symbol6_normalization_len_le1 shows that each symbol 6 step emits at most one normalization byte;
- all_six_first_four_no_normalization and all_six_normalization_budget combine zero-byte emission in the first four steps with a total upper bound of 1020 bytes;
- all_six_trace_fits_mode2 adds the final four state bytes, placing total trace size in $[4, 1024]$.

The existing encoder closure is strengthened by actual_rans_encode_failure_trace_cause, which concludes that nonzero *bad* implies a normalization trace exceeding 1020 bytes. Contradiction with the all-6 bound gives the following actual postconditions.

Verified proposition 4.23 (**OBL-RANS-ACTUAL-SUCCESS-WITNESS**)**.** actual_rans_encode_all_six_success establishes $bad = 0$, $0 \le off \le 1020$, $4 \le 1024 - off \le 1024$, an exact trace segment, and a prefix frame for every returning execution of actual `_rans_encode`. actual_encode_hb_z1_full_zero_success and signature_pack_hbz_zero_success_mode2 lift this to the focused/production full encoder. signature_pack_unpack_hbz_zero_success_mode2 concludes the production zero round trip, `decoder_ran`, decoder $bad = 0$, and coefficient/encoded-tail frames.

This proof does not require success as a precondition, but remains partial correctness in Hoare logic. It includes no assertion that an actual fixed-input execution terminates or exists, no losslessness or probability-one success, and no `phoare` theorem. Both "fixed all-6 Hoare success is **PROVED**" and "fixed-input termination is unproved" must therefore be retained.

4.17 Stage 16: the frozen actual KeyGen snapshot and KG–NTT stopping boundary

Week 16 added 8 MINCORE targets: three for KeyGen, one for Sign, and four for Verify [23, 22, 21, 24, 25]. In the first pure file, `Mode2KeygenSnapshotAlgebra.ec`, snapshot_residue_exact_low_high, raw_sum_raw_residue_congruent_mod_2q, and snapshot_expression_from_product_congruent_mod_2q prove low/high decomposition of canonical residues and the doubled congruence.

File `Mode2KeygenCoreEquation.ec` defines the transparent harness

```
ActualM23MatrixFinalizeSnapshot.run.
```

It snapshots the return of actual `_kp_m23_matrix(2,3)`, then calls actual `_keypair_finalize_m23(512)`.

Verified proposition 4.24 (Actual snapshot/finalizer boundary)**.** actual_m23_matrix_finalize_snapshot composes the matrix-output/NTT representation, exact finalization, and tail frame. The strongest theorem, actual_m23_matrix_finalize_semantic_snapshot, adds finalize_semantic_output_low_high_decomposition, HAETAE finalizer semantics, and finalize_semantic_output_snapshot_mod2q_zero, yielding a KG-2-like decomposition, adjusted s_2, and a snapshot-only zero congruence mod-$2q$ at every active coefficient.

The third target, `Mode2KeygenNttMulBridge.ec`, fixes only the last sound rewrite in the existing NTT chain, without duplicating actual loops. Under the transformed-secret representation, output_row_from_mode2_ntt_words proves that `output_row` equals exactly

$$\text{array256_mont}(\text{full_invntt}(\text{pointwise_row_words}(m, v, row)))$$

output_row_repr_from_mode2_ntt_words transports this equality to the existing `wide_slice_repr_bound` contract. actual_m23_matrix_snapshot_rows_explicit reuses the strongest snapshot theorem by consequence, reducing both active rows to that full-invNTT/pointwise-word representation. None of these theorems assumes a native R_q product, security-model product, or the desired KeyGen equation.

Open obligation 4.25 (Frozen **OBL-MINCORE-KEYGEN**: `STOP-KG-NTT`)**.** At the frozen-paper stage, the final NTT-word rewrite compiled, but the odd-root orthogonality/full-NTT convolution theorem converting actual pointwise/inverse-NTT representation to native R_q negacyclic multiplication was missing, as was the adapter transporting `Rq.poly` to the security model's integer-list product. The row product $\sum_j A_{\text{gen},ij} s_{\text{gen},j}$ was therefore unavailable. Faithful KG-1, KG-3, complete KG-4, and the paper equation $As = qj \pmod{2q}$ were not proved.

The KG-NTT-MUL audit identified these two missing leaves and ended at `STOP-KG-NTT`, not `GO-KG`. KeyGen was frozen at the KG-2/finalization boundary. Sampling, retries, packing, the public API, KeyGen termination, and security lie outside the conclusions of this actual two-call snapshot theorem. Sign was subsequently frozen at `STOP-SIGN-CHAL-MODE2`, and Verify at `STOP-VERIFY-MATRIX-CRT` [24, 25, 4].

4.18 Stage 17: post-freeze NTT/Rq transport and the actual avec/qj boundary

Post-freeze work closed the first algebraic gap above without expanding the frozen 82-target manifest. In `NTTFullSpectralAction.ec`, rq_mul_coeff_foldr_to_bigi and full_ntt_montgomery_spectral_action connect the implementation-style negacyclic coefficient sum to full-NTT spectral action. full_ntt_montgomery_row_product composes this into a row product for arbitrary column counts; `NTTMode2RowSpecializations.ec` provides pure specializations for KeyGen's three columns and Verify's four. This Verify result is not a theorem about actual `_polyvec_ntt`/`_polymat_pointwise_acc`/ `_polyvec_invntt` procedures and does not close frozen `STOP-VERIFY-MATRIX-CRT`.

rq_poly_add_repr, rq_poly_mul_repr, and rq_poly_dot3_repr in `RqHAETAEBridge.ec`, together with keygen_mode2_row_product, mode2_row_product_haetae_dot, and actual_m23_matrix_snapshot_rows_row_product in `Mode2KeygenNttMulBridge.ec`, transport the native R_q three-term row product to the HAETAE `poly_dot` representation [12].

Verified proposition 4.26 (Deterministic boundary of actual avec and the paper's a, qj)**.** `KgActualAvecQjSemantics.ec` fixes mode 2 row nonces 515/516, $[3 + row, 2]$ after the 32-byte seed, SHAKE128 domain/padding 31/128, 168-byte blocks, little-endian 16-bit parsing,

and < 64513 rejection. mode2_uniform_stream_paper_a_flat interprets the two 256-coefficient rows of the terminating actual sampler as the paper's a. The paper's $j = (1, 0)^T$ is separated through paper_qj_row0_coeff0, paper_qj_row0_other, and paper_qj_remaining_row. checked_mode2_parent_m23_finalize_actual_avec_qj combines current actual-sampler, matrix-helper, and finalizer results into one Hoare boundary. This is partial correctness of terminating executions, not a distribution, uniformity, losslessness, or termination claim.

Verified proposition 4.27 (Faithful augmented coefficient-product composition)**.** The latest unmanifested standalone file `KgFaithfulAugmentedModel.ec` instantiates a width-6 carrier with actual arrays through `faithful_augmented_matrix_coeff`, `faithful_augmented_secret_coeff`, `actual_faithful_augmented_matrix`, and `actual_faithful_augmented_secret`. `faithful_middle_matrix_poly` and `faithful_middle_secret_poly` reconstruct the carrier's middle three columns as HAETAE polynomials, so there is no free generated-block argument detached from the displayed matrix/secret. It then composes actual_snapshot_paper_qj_to_faithful_augmented, generated_product_coeff_from_rq_repr, faithful_augmented_As_qj_from_generated_repr, rq_as_haetae_poly_repr, faithful_generated_row_product_haetae_dot, actual_faithful_augmented_productE, faithful_augmented_product_qj_from_generated_dot, and mode2_m23_facts_generated_row_product. The strongest theorem, checked_mode2_parent_m23_finalize_faithful_augmented_paper_as_qj, proves congruence mod-$2q$ between `faithful_augmented_matrix_vector_product_coeff` and `faithful_qj_coeff` at each active coefficient of the actual checked sampler/matrix/finalizer parent. The file passed standalone `-no-eco` fresh compilation.

The exact endpoint is a structured coefficient product specific to the actual carrier. `actual_faithful_augmented_productE` closes the expansion of the product taking only matrix and secret as inputs. The middle 2-by-3 block is reconstructed from the specified columns and computed with HAETAE `poly_dot`. The verdict for this work is `GO-KG-FAITHFUL-AUGMENTED`. These dedicated mod-$2q$ block semantics are not a coefficient theorem for the existing width-4, mod-q security `matrix_vec_mul`, nor do they identify security-side synthetic ExpandVecA with the actual sampler. [11]

Open obligation 4.28 (`STOP-KG-AVEC-QJ`: **OBL-KG-SECURITY-EXPANDVECA-SEMANTICS**)**.** The existing security-side `haetae_mode23_qj_vector` is a synthetic object using `public_key_seed_polyveck` tag 401. security_named_qj_is_not_paper_qj proves that its row 0 coefficient 0 is 401, whereas the paper's qj has $q = 64513$. The zero-seed execution also has actual nonce-515 first coefficient 44985. The security specification must therefore be replaced or refined with actual SHAKE128/LE16/rejection/nonce semantics, keeping the paper's qj as a separate correction, and an actual-array representation theorem must then be proved. The paper-shaped congruence for the new custom carrier cannot therefore be promoted to KG-1–KG-4 of the existing security model or security-level $As = qj \pmod{2q}$ [10].

4.19 Exact endpoints of the completed chains

Chain	Endpoint
packed key → generated raw μ → position-64 absorb	**PROVED** (raw/internal, local)
actual KeyGen export, Sign/Verify import, address/frame/control	**PROVED** (each named contract)
direct equality of stored μ in completed actual traces	**PARTIAL**
canonical signature prefix round trip	**PROVED** (1056-byte prefix)

Chain	Endpoint
HBZ leaf, table, pure rANS step and byte trace	**PROVED** (pure/leaf)
actual encoder array/list, word, and serialization leaves	**PROVED** (Week 10 local propositions)
actual encoder inner byte/frame and successful-output size	**PROVED** (exact 0..2, actual partial correctness)
actual encoder entire trace-suffix refinement	**PROVED** (success-conditioned partial correctness)
actual decoder exact-trace refinement	**PROVED** (1024-symbol recovery, exact consumption, tail frame)
actual encoder–copy–decoder harness control	**PROVED** (calls/branches only)
actual encoder–copy–decoder inverse	**PROVED** (success-conditioned partial correctness)
production full-HBZ wrapper inverse	**PROVED** (success-conditioned partial correctness, production boundary)
actual encoder success witness	**PROVED** (Hoare failure exclusion for fixed all-6 input; no termination/losslessness)
actual KeyGen matrix/finalize snapshot	**PARTIAL** (KG-2-like decomposition and snapshot congruence proved; frozen paper ends at `STOP-KG-NTT` after the final NTT-word rewrite)
post-freeze full-NTT/native R_q/HAETAE row-product transport	**PROVED** (named pure/representation theorems outside the frozen 82 targets)
post-freeze actual `avec` and paper a, qj boundary	**PARTIAL** (deterministic semantics and separate qj composition proved; `STOP-KG-AVEC-QJ` at the security-object mismatch)
faithful augmented coefficient-product/transport	**PROVED** (standalone, outside the frozen 82 targets; actual-carrier-specific product, not generic security-model `matrix_vec_mul` equivalence)
actual accepted Sign core	**PARTIAL** (call/branch control only; `STOP-SIGN-CHAL-MODE2`)
actual Verify core	**PARTIAL** (helper-local word/control only; frozen `STOP-VERIFY-MATRIX-CRT`)
h suffix codec	**DEFERRED** (Week 16 MINCORE KeyGen scope)
full signature codec and Sign–Verify correctness	**BLOCKED**

Chain	Endpoint
implementation-level functional correctness and security	**SPECIFIED**/ **PARTIAL**

5 The HBZ/rANS dynamical system and fixed-input failure exclusion

5.1 Why success postconditions and termination are different claims

The inverse function for the signature prefix, the HBZ coefficient–symbol leaf, the edge mapping canonical symbol streams through the actual encoder to pure byte traces, the edge mapping exact traces through the actual decoder back to canonical symbol streams, core composition across an actual copy, and the full-HBZ wrapper boundary of production SignaturePack/Unpack have each been closed. Week 15 excluded failure of the actual encoder when it returns on the all-6 stream produced by canonical all-zero HBZ, and lifted this to a production zero round trip [20, 8]. Thus **OBL-RANS-ACTUAL-SUCCESS-WITNESS** is **PROVED** as fixed-input Hoare partial correctness. It is not a termination/losslessness theorem guaranteeing a return or the existence of an execution. After this result, the project moved through the frozen Week 16 KeyGen/Sign/Verify `STOP-VERIFY-MATRIX-CRT` to the `STOP-KG-AVEC-QJ` of the post-freeze KeyGen semantics audit.

The integer arithmetic of rANS itself is not complicated. The difficulty lies in implementation direction: the encoder reads symbols backwards and decrements the byte-buffer cursor, while the decoder reads the copied buffer forwards. The key is an invariant connecting the pure inverse-function theorem to the states of the two actual nested loops. Week 11 completed the reverse-encoder connection, Week 12 the forward-decoder connection, and Week 13 the actual copy and premise transport between the two theorems. Week 14 composed this core theorem with the prepare/apply, frame and size semantics of production `_encode_hb_z1_full`/`_decode_hb_z1_full`. Week 15 combines the fact that actual prepare produces an all-6 stream, a 1020-byte normalization budget, and a necessary condition for actual failure to force the nonzero-size branch in returning fixed-input executions. The remaining HBZ gaps are termination and existence of this execution itself, global success on arbitrary canonical inputs, malformed rejection, and probabilistic losslessness.

5.2 The 13-symbol partition

Mapping a canonical HBZ coefficient $h_i \in [-6, 7)$ to $s_i = h_i + 6$ gives the alphabet

$$\Sigma = \{0, 1, \dots, 12\}$$

Let the rANS scale be $M = 1024$, and let each symbol's cumulative starting point and frequency be

$$(c_s)_{s=0}^{12} = (0, 1, 2, 3, 8, 66, 312, 710, 957, 1016, 1021, 1022, 1023),$$
$$(f_s)_{s=0}^{12} = (1, 1, 1, 5, 58, 246, 398, 247, 59, 5, 1, 1, 1)$$

The certificate establishes

$$f_s > 0, \qquad \sum_{s=0}^{12} f_s = M, \qquad [c_s, c_s + f_s) \text{ partition } [0, M)$$

This finite partition uniquely recovers the symbol from the residue slot $y \bmod M$. The fact that the actual encoder/decoder's packed tables represent exactly these c_s, f_s values and the symbol lookup is also certified against the generated literal arrays.

5.3 The pure step is an inverse of Euclidean division

For a fixed symbol s, define

$$E_s(x) = \left\lfloor \frac{x}{f_s} \right\rfloor M + c_s + (x \bmod f_s),$$
$$D_s(y) = \left\lfloor \frac{y}{M} \right\rfloor f_s + (y \bmod M) - c_s$$

Euclidean division $x = f_s\lfloor x/f_s \rfloor + (x \bmod f_s)$ and $0 \le x \bmod f_s < f_s$ immediately suggest

$$D_s(E_s(x)) = x$$

The EasyCrypt theorems pure_rans_step_quotient, pure_rans_step_slot, and pure_rans_step_inverse prove these calculations separately.

The actual encoder uses a fast formula with table reciprocals, shifts and biases instead of division. A separate certificate shows that under

$$1 \le x < x_{\max}(s), \qquad x_{\max}(s) = 2^{21} f_s$$

the fast quotient equals the Euclidean quotient, so D_s also inverts the fast step. This is not merely a computational check that "the table values look right"; it is a theorem connecting the integer meanings of the actual word fields.

Verified proposition 5.1 (Pure and fast steps)**.** For canonical symbols $0 \le s < 13$ and the specified state ranges,

$$D_s(E_s(x)) = x, \qquad D_s(E_s^{\text{fast}}(x)) = x$$

compile successfully. There is also a concrete witness to the premises at $s = 0, x = 1$, so the theorem is not vacuous.

5.4 Normalization bytes are coordinates of the discarded quotient

When the state exceeds $x_{\max}(s)$, divide it by the byte radix $B = 256$ 0, 1 or 2 times to reduce it before applying the fast step. The pure definitions are

$$\ell_s(x) \in \{0, 1, 2\}, \qquad r_s(x) = \left\lfloor \frac{x}{B^{\ell_s(x)}} \right\rfloor$$

and the low bytes discarded by division are retained in a list $\beta_s(x)$ in decoder read order.

The key readback proposition is

$$\text{ReadBytes}(r_s(x), \beta_s(x)) = x.$$

For one byte, use $x = B\lfloor x/B \rfloor + (x \bmod B)$; for two bytes, apply this identity twice. The actual encoder writes the low byte first while decrementing the cursor, so the final forward segment orders the two bytes as $[\mathsf{high}, \mathsf{low}]$. Missing this reversal can leave a pure theorem correct while making it inconsistent with the actual decoder.

`Mode2RansNormalization.ec` provides leaf theorems showing that the actual $W32$ shift/truncate/or operations and $W64$ cursor decrement have the same meanings as these integer operations.

5.5 A pure trace for the complete symbol list

Let the symbol list be $s_0, \dots, s_{n-1}$, with $n = 1024$, and let the final state be $x_n = L = 2^{23}$. Define recursively, so that the encoder proceeds backwards,

$$\begin{aligned} \ell_j &= \ell_{s_j}(x_{j+1}), \\ r_j &= \left\lfloor x_{j+1}/256^{\ell_j} \right\rfloor, \\ x_j &= E^{\mathrm{fast}}_{s_j}(r_j), \\ \beta_j &= \beta_{s_j}(x_{j+1}) \end{aligned}$$

The decoder-facing byte string is

$$\mathcal{B} = \mathrm{LE32}(x_0) \| \beta_0 \| \cdots \| \beta_{n-1}$$

Define the cursor cuts by

$$c_0 = 4, \qquad c_{j+1} = c_j + |\beta_j|$$

The project names *xs*/*cuts*/*bytes* refer respectively to $(x_0, \dots, x_n)$, $(c_0, \dots, c_n)$, and $\mathcal{B}$. These three objects are defined in `Mode2RansByteStack.ec`. Its `valid_rans_trace` constructs them directly from the canonical symbol list rather than assuming arbitrary witnesses.

Verified proposition 5.2 (Inductive inversion of the trace)**.** For a canonical list,

$$2^{23} \le x_j < 2^{31}$$

holds for every j. Moreover, the residue slot of x_j selects s_j, $D_{s_j}(x_j) = r_j$, and reading β_j recovers x_{j+1}.

The mathematics required for the pure decoder's list-level inductive proof is therefore available. The refinement showing that actual encoder states represent these trace coordinates was closed in Week 11; the refinement showing that actual decoder states consume the same coordinates forwards was closed in Week 12.

5.6 The six encoder targets audited in Week 10

Week 10 decomposed the required outer-loop invariant into distinct transport problems and added 6 files to the authored target manifest. All six files compile freshly with `-no-eco`, but completion of each local proposition must be distinguished from completion of the parent refinement.

- `Mode2RansArrayListBridge.ec`: transports the actual byte array's symbol stream to a pure list and suffix, and expresses segment agreement and the prefix frame. The array–list bridge and one-symbol trace recurrence are **PROVED**.
- `Mode2RansEncoderWordStep.ec`: connects actual word expressions using packed reciprocals, shifts and complements to the integer meaning of the pure fast step. Word-level equalities for the literal mode 2 table are **PROVED**.
- `Mode2RansEncoderInnerProgress.ec`: represents the pure 0/1/2-step progress of the normalization inner loop through the state, cursor and byte segment already written. This pure lemma is **PROVED**.
- `Mode2RansEncoderSerialization.ec`: separates the four little-endian bytes of the final $W32$ state, the exterior frame and successful size arithmetic. This leaf proposition is **PROVED**.

- `Mode2RansEncoderTrace.ec`: supplies lemmas for finite phases, byte prepending, segments, prefix frames and absence of wrap used by the actual inner loop.
- `Mode2RansEncoderActualInner.ec`: installs these invariants in the actual generated `_rans_encode` inner loop and derives successful offset and size ranges through a separate actual Hoare theorem.

Caution 5.3 (Local **PROVED** and the then-**PARTIAL** parent in Week 10)**.** At the Week 10 baseline, these files certified only lemmas and local actual partial correctness. Equality of the complete actual suffix with the pure `trace_bytes` was absent, so **OBL-RANS-ENCODE-REFINEMENT** was **PARTIAL**. The next section closes exactly this parent edge in Week 11. These Week 10 targets did not certify success branch reachability or termination. Fixed all-6 failure exclusion is a separate Week 15 theorem, and termination remains open.

5.7 The actual encoder invariant closed in Week 11

Let i be the actual reverse encoder's outer-loop index, x its current word state, off the buffer cursor decreasing from the end, and enc the buffer. With the pure trace

$$T_i = \mathrm{EncodeTrace}(\mathrm{SymbolSuffix}(symbols_0, i)) = (x_i, b_i)$$

the success branch's live invariant consists of the following five conditions [7].

$$0 \le i \le 1024, \qquad x = x_i, \qquad off + |b_i| = 1024,$$
$$\mathrm{SegmentMatches}(enc, off, b_i), \qquad \mathrm{PrefixFrame}(enc_0, enc, off), \qquad 4 \le off.$$

Week 11 added 7 targets. The tail-aware inner invariant preserves the initial array enc_0 and $\mathrm{EncodeTrace}(tail).bytes$ across actual byte writes and recovers the exact 0..2 normalization length when the actual guard exits. Generated word-step lemmas identify the word update after actual table loads and protect erasure with the pure fast step; the final four generated stores prepend the serialized state to the accumulated suffix.

Consequently, the actual procedure's success postcondition compiled as

$$\mathrm{SegmentMatches}(enc', off', \mathrm{TraceBytes}(\mathrm{SymbolList}(symbols_0))),$$
$$off' + |\mathrm{TraceBytes}(\mathrm{SymbolList}(symbols_0))| = 1024,$$
$$\mathrm{PrefixFrame}(enc_0, enc', off')$$

Verified proposition 5.4 (**OBL-RANS-ENCODE-REFINEMENT**)**.** actual_rans_encode_trace_closure shows that if the actual generated `_rans_encode` returns, it either fails or, on success, satisfies all three relations above, the offset/size ranges and $bad = 0$. actual_rans_encode_trace_refinement explicitly exposes equality of the entire returned suffix at the parent refinement surface. Success is not a premise; it is one branch of the conclusion determined by the actual return value.

Caution 5.5 (Limits of partial correctness)**.** This theorem alone does not establish actual encoder termination, success reachability or an all-6 symbol success witness. In Week 11, **OBL-RANS-ACTUAL-SUCCESS-WITNESS** was **PARTIAL**; the separate Week 15 theorem excludes only failure in returning fixed all-6 executions.

5.8 The actual decoder invariant closed in Week 12

For actual forward decoder index i, state x and read cursor off_d, Week 12 installed the following invariant directly in the generated outer/inner loops [17, 6].

1. The symbol prefix already output equals $s_0, \ldots, s_{i-1}$;
2. the current state is x_i and the current cursor is c_i;
3. table lookup selects s_i and reads the normalization segment exactly from $[c_i, c_{i+1})$;
4. the frame of the unwritten tail of the output array is preserved;
5. at exit, $x = 2^{23}$, $off_d = size$, and $bad = 0$.

The historical actual_rans_decode_mode2_control fixed only the actual tables/state fields and $bad \in \{0, 1\}$. The new input predicate `actual_mode2_decoder_trace_input` combines the expected symbol array, exact `trace_bytes`, pointwise relations for every normalization read, state fields $(1024, encoded_size, 13)$, and actual generated tables.

Verified proposition 5.6 (**OBL-RANS-DECODE-REFINEMENT**)**.** actual_rans_decode_trace_refinement directly opens the actual generated `_rans_decode` and, under this exact-trace input, recovers all 1024 symbols and concludes $off = encoded_size$, $bad = 0$, and the decoded tail frame. decoder_outer_trace_exit_components internally derives $x = 2^{23}$, also satisfying the actual final-state rejection check.

This is Hoare partial correctness and does not establish decoder termination. The exact-trace input is non-circular; Week 13 added a separate composition theorem showing that the actual harness produces it.

5.9 Actual harness core composition closed in Week 13

The harness in `Mode2RansActualInverse.ec` is not an abstract assembly diagram: it calls three actual generated procedures in the following order. The theorem in that file closes only control of these calls; the Week 13 semantic closure is separate, in `Mode2RansCoreActualInverse.ec`.

$$\mathsf{RansEncode} \longrightarrow \mathsf{CopyEncodedSuffix} \longrightarrow \mathsf{RansDecode}.$$

The latter two calls run only when the encoder returns $bad = 0$.

The existing harness control conclusion is

$$\begin{aligned} &encoderBad \in \{0, 1\}, \\ &decoderRan \iff encoderBad = 0, \\ &decoderRan \implies (count, size, m) = (1024, encodedSize, 13) \end{aligned}$$

Week 13's `Mode2RansCoreCompositionBridge.ec` composes the encoder's `segment_matches` and the copy's `slice_eq` into an exact trace in the copied buffer, and transports it to the decoder's pointwise-read and configured-state input predicate. In particular, it derives absence of wrap from actual success bounds instead of silently replacing modular (W64) subtraction with integer subtraction.

Verified proposition 5.7 (**OBL-RANS-CORE-INVERSE**)**.** actual_rans_encode_copy_decode_inverse does not assume actual encoder success. On failure, the actual copy and decoder are not run, and the initial decoded array/state is preserved. On success, the following semantic conclusions appear in the postcondition of the same harness theorem:

$$decoderBad = 0, \quad off_{\text{final}} = size, \quad decoded[0..1024) = symbols[0..1024).$$

The decoder subproof also derives the internal exit state $x = 2^{23}$, but the harness ABI does not expose it. The decoded tail $[1024, 2048)$ is preserved by a frame. This result is success-conditioned partial correctness.

At the end of Week 13, no EasyCrypt theorem forced the actual success branch. A concrete pure-trace witness, a canonical all-zero precondition witness, or a size-4 decoder-state witness did not replace the postcondition that the actual encoder returns $bad = 0$ on the concrete all-6 input. Week 15 closes exactly this gap by contradicting the budget with the actual failure cause.

5.10 Correctly reading the full HBZ wrapper closed in Week 14

Because the actual encoder can express failure, canonical h alone must not be used to assert an unconditional round trip. The parent proposition proved in Week 14 must expose the failure branch, as in

$$size = 0 \quad \vee \quad (size \neq 0 \wedge bad = 0 \wedge decoded[0..1024) = h[0..1024))$$

Nonzero size bounds and buffer frames are also required.

Verified proposition 5.8 (**OBL-SIG-HBZ-ENCODE-DECODE**)**.** actual_hbz_full_encode_decode_inverse_mode2 proves this failure/success disjunction for the focused actual full-HBZ harness. signature_pack_unpack_hbz_full_actual_exact shows that the production SignaturePack/Unpack harness agrees exactly with the focused harness in its return tuple and `decoder_ran` flag, and signature_pack_unpack_hbz_full_inverse_mode2 lifts the same contract to the production boundary. The success branch includes $4 \leq size \leq 1024$, coefficient-prefix recovery, coefficient/suffix frames, and a prepared-symbol trace witness.

5.11 Week 15's fixed all-6 budget and actual failure exclusion

Opening byte projections to prove that all actual 32-bit loads from `zero_hbz` are 0 shows that actual prepare returns $bad = 0$ and a prefix of 1024 symbol 6 entries. In the literal table, $f_6 = 398$ and $x_{\max}(6) = 2^{21} \cdot 398$; symbol6_normalization_len_le1 shows that each symbol 6 step with $2^{23} \leq x < 2^{31}$ emits at most one normalization byte.

The first four states computed from the initial state are

$$8388608 \to 21582496 \to 55528910 \to 142868116 \to 367580518$$

and all four steps emit 0 bytes. Thus, even allowing one byte for each of the remaining 1020 steps gives normalization length at most 1020; adding the final four state bytes puts the total trace in $[4, 1024]$. This is a conservative capacity argument, not a claim that the observed exact 174/178-byte execution has been proved as a theorem.

Verified proposition 5.9 (**OBL-RANS-ACTUAL-SUCCESS-WITNESS**)**.** actual_rans_encode_failure_trace_cause states that a nonzero *bad* from the actual encoder implies trace length $>$ 1020. Contradicting this with the all-6 upper bound lets actual_rans_encode_all_six_success establish $bad = 0$ and the size/frame/exact-trace postconditions for every returning actual core execution. actual_encode_hb_z1_full_zero_success, signature_pack_hbz_zero_success_mode2, and signature_pack_unpack_hbz_zero_success_mode2 lift this conclusion to the focused full wrapper including actual prepare and to the production SignaturePack/Unpack zero round trip.

Caution 5.10 (`GO-WITNESS` is not `GO-WITNESS-LOSSLESS`)**.** These are Hoare partial-correctness theorems excluding failure in every *returning* fixed-input execution. Fixed-input termination, execution existence (non-vacuity), losslessness, probability-one success and `phoare` have not been proved. The h codec became **DEFERRED** because work moved to Week 16 MINCORE KeyGen, not because the success witness was incomplete.

rANS layer	Status
13-symbol intervals/frequencies and actual table certificate	**PROVED**
pure quotient/slot/fast-step inverse	**PROVED**
0/1/2-byte normalization and W32/W64 leaf semantics	**PROVED**
canonical *xs*/*cuts*/*bytes* trace and state bounds	**PROVED**
actual array/list suffix and word-step bridge	**PROVED**
actual four-byte final serialization composition	**PROVED**
actual encoder successful offset/size ranges	**PROVED** (partial correctness)
actual inner normalization byte/frame/no-underflow	**PROVED** (tail-aware exact 0..2 exit)
actual encoded suffix copy and tail frame	**PROVED**
actual encoder–copy–decoder branch control	**PROVED** (control only)
actual encoder produces the complete pure trace	**PROVED** (success-conditioned partial correctness)
actual decoder consumes the exact trace	**PROVED** (1024-symbol recovery, exact consumption, tail frame)
actual encoder–copy–decoder core inverse	**PROVED** (success-conditioned partial correctness)
full HBZ encode/decode	**PROVED** (production-wrapper success-conditioned partial correctness)
actual encoder success witness	**PROVED** (fixed all-6 Hoare failure exclusion; no termination/losslessness)

6 Open boundaries and limits on claims

6.1 The narrow but important semantic gap: product/replay

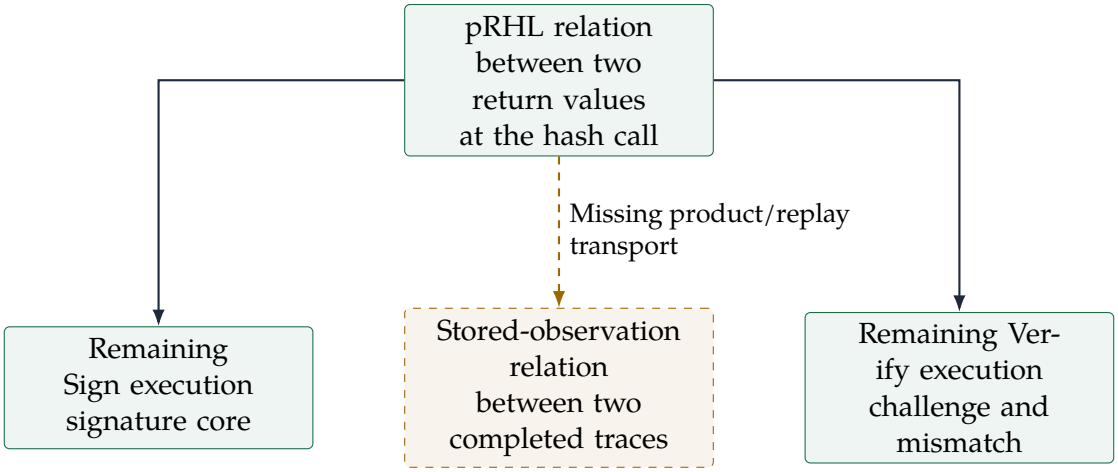
The raw μ line currently contains two separate facts.

1. The *return values* of the actual generated Sign/Verify hash calls have the same 32-byte prefix on related inputs;
2. the actual complete Sign-then-Verify execution corresponds exactly to a ghost trace, which stores each hash result in `observed_mu`.

The desired conclusion in an accepted Verify post-state is

$$\mathit{VerifyResult} = 0 \Longrightarrow \mathrm{MuPrefix}_{32}(\mathit{SignTrace.observed_mu}, \mathit{VerifyTrace.observed_mu})$$

Mathematically, combining facts 1 and 2 may appear immediate, but the variables in the two propositions do not coexist at the same point in the current program-logic surface.



The hash-call pRHL goal contains $\mathit{res}\{1\}, \mathit{res}\{2\}$, but those local return variables disappear from the unary post-state after the entire trace completes. Conversely, applying the pRHL theorem at the hash-call point requires handling the different remaining continuations of Sign and Verify. Discarding the Sign continuation on one side requires losslessness or an appropriate product rule, neither of which is yet available.

Open obligation 6.1 (**OBL-MU-PRODUCT-REPLAY**)**.** A sound rule or proof structure is needed to transport the return-value-level region-local pRHL theorem to a stored-observation relation in an already completed sequential trace.

This obligation was frozen in Week 7 as **DEFERRED**/ an explicit paper boundary [26]. The following approaches were deliberately rejected.

- Assuming observation equality as a premise and restating the conclusion;
- adding a new SHAKE locality axiom;
- continually enlarging the theorem surface with another caller/observation wrapper;
- discarding the Sign continuation without a termination justification.

Accordingly, $\delta_{\mu,\mathrm{raw_api_accept}} = 0$ is not currently claimed. This semantic gap arises from a missing program-logic transport arrow, not from a lack of byte-locality theorems.

6.2 Accepted-path and legitimate-signature correctness run in different directions

The direction proved for actual Verify is

$$\text{Accept} \Longrightarrow \text{TailReached} \Longrightarrow \text{HashInputsBound}$$

This is a security-facing lane analyzing executions that have already accepted.

The direction required for functional correctness is

$$\sigma \leftarrow \text{Sign}(sk, M) \Longrightarrow \text{Verify}(vk, M, \sigma) = \text{Accept}$$

This requires showing that signatures produced by Sign pass every unpacking, norm, hint, and challenge gates. A theorem in the first direction cannot be reversed into one in the second direction.

Open obligation 6.2 (**OBL-SIGN-OUTPUT-TAIL-REACH**)**.** Actual Sign output must be proved to pass actual Verify's early-reject tree and reach the tail. The signature-prefix inverse and success-conditioned full-HBZ wrapper are closed, and failure is also excluded for returning fixed all-zero HBZ executions. However, this fixed input does not substitute for canonicality of all actual Sign output. The h suffix, metadata/padding, norm, hint and arithmetic correctness remain.

OBL-SIGN-VERIFY-CORRECTNESS is **BLOCKED** by this obligation and the remaining arithmetic and challenge correctness obligations.

6.3 Representation obligations remaining in the full signature codec

Extending the prefix's section–retraction to the full codec requires at least the following squares. Week 14 closed one HBZ edge among them, and Week 15 excluded failure of returning fixed all-zero executions on that edge.

- The two size-delta metadata bytes denote the same sizes in pack/unpack;
- the actual full HBZ rANS wrapper is an inverse under the success condition (**PROVED**, Week 14);
- fixed-input Hoare success for canonical all-zero HBZ (**PROVED**, Week 15; termination is unproved);
- encode/decode for the second, h, suffix codec (**DEFERRED**, transition to Week 16 MINCORE);
- canonicality of trailing zero padding and the parser's zero gate;
- preservation of array tails outside the target by the packer/unpacker;
- actual Sign produces canonical inputs and successful buffer bounds.

Thus **OBL-SIG-PREFIX-CODEC** is **PROVED** in a limited sense, while **OBL-SIG-FULL-CODEC-MODE2** is **BLOCKED**. The prefix inverse and full inverse must not be described as the same proposition.

6.4 Remaining algebraic, numerical and probabilistic obligations

The frozen paper's MINCORE frontier was recorded as KeyGen `STOP-KG-NTT`, Sign `STOP-SIGN-CHAL-MODE2`, and Verify `STOP-VERIFY-MATRIX-CRT`. Post-freeze work closed

`rq_mul_coeff_foldr_to_bigi`, full-NTT spectral action, native R_q row products and HAETAE representation transport, but a semantic mismatch between actual ExpandVecA and the existing security-side synthetic object creates `STOP-KG-AVEC-QJ`. Separately, the actual Verify 2-by-4 NTT/CRT/freeze procedure semantics remain a frozen obligation. Full implementation refinement still requires the following independent obligations.

public-key algebra/NTT
: R_q-module computations, NTT representations and the public-key relation agree with the actual implementation.

sampler distribution
: The output distributions of uniform/small/hyperball samplers and the actual random tape agree with the paper distributions or have explicit distance bounds.

rejection and termination
: KeyGen/Sign retries terminate, and the accepted distribution and retry loss are calculated.

highbits/LSB
: Actual coefficient decomposition, rounding and packing agree with the paper's integer decomposition, and both challenge inputs are equal.

FIPS202 transcript
: Every domain separator, length, absorb order and squeeze length agrees byte-for-byte with the fixed specification.

FFT tie policy
: Fixed-point FFT, including boundary tie handling, is connected to the analytic predicate.

two-transcript extraction
: The actual byte-level challenge and accepted signature instance are connected to two-transcript extraction in the paper/ROM reduction.

6.5 The KG–NTT–MUL stopping boundary of frozen Week 16 KeyGen

The transparent two-call harness for actual `_kp_m23_matrix` and `_keypair_finalize_m23` places the pre-finalization *bp* snapshot, finalizer semantics, KG-2-like low/high decomposition, adjusted s_2, and a snapshot-only mod-$2q$ zero congruence in the same postcondition [23, 22, 21]. This is meaningful **PARTIAL** progress at the actual coefficient boundary of **OBL-MINCORE-KEYGEN**.

The matrix helper's result is expressed as `KeygenM23ArithmeticSpec.output_row`. The new output_row_from_mode2_ntt_words rewrites this exactly in terms of actual full NTT, pointwise row products, inverse NTT and Montgomery representation; actual_m23_matrix_snapshot_rows_explicit applies that expression to the transparent harness's two active rows. At the frozen-paper point, there was neither a compiled theorem equating this spectral expression with native R_q negacyclic multiplication nor an adapter transporting `Rq.poly` into the security model's $A_{\text{gen}}s_{\text{gen}}$ representation. The KG-NTT-MUL audit identified odd-root orthogonality/full-NTT convolution and this adapter as the missing leaves and ended with `STOP-KG-NTT`. The frozen verdict prohibits the following promotions.

- Renaming snapshot *pre_bp* to $A_{\text{gen}}s_{\text{gen}}$ without justification;
- concluding faithful KG-1 or KG-3 from KG-2-like coefficient decomposition;
- concluding a complete KG-4 vector from one adjusted s_2 component;
- promoting the snapshot congruence to the paper equation $As = qj \pmod{2q}$;

- inferring sampler, retry, packing, public API or KeyGen termination results from this two-call harness.

6.6 Post-freeze KG progress and the new semantic boundary

Post-freeze shared NTT theory proved pure coefficient normalization, full-NTT spectral action and row products with arbitrary column counts. In Mode 2, pure specializations for KeyGen's 3 columns and Verify's 4 columns both compile. The KeyGen side was transported from native R_q row products to `HAETAE_Algebra.poly_dot` [12]. However, the Verify 4-column theorem does not open the actual helper loop and therefore does not clear `STOP-VERIFY-MATRIX-CRT`.

The next audit connected actual `avec`'s SHAKE128 framing, nonces 515/516, little-endian candidates and < 64513 rejection to paper a, and independently added paper $qj = q(1,0)^T$ to the snapshot zero equation. However, the following three objects are distinct.

actual SHAKE128/rejection $a \neq$ synthetic security generator, paper qj is a separate correction.

A compiled counterexample shows that security-named row 0 coefficient 0 is 401, whereas paper $qj[0,0] = 64513$. The actual value in the all-zero seed trace is also 44985. Repairing `STOP-KG-AVEC-QJ` therefore requires replacing or refining synthetic `public_rounding_vector_seed` with actual expansion semantics and proving a coefficient-wise actual-array representation [10]. Before crossing this boundary, the following promotions are also prohibited.

- Renaming the synthetic object to actual `avec` or paper qj;
- assuming that the separate qj correction is preserved in a modulo-q HAETAE vector;
- renaming the new coefficient-product dedicated to the actual carrier as a security-level width-6 mod-$2q$ equation without a theorem for general ring `matrix_vec_mul` defined solely from the existing security matrix/secret;
- inferring sampler distribution, uniformity, losslessness or termination from deterministic partial correctness.

`KgFaithfulAugmentedModel.ec` defines an actual width-6 carrier and generated-product transport for this packaging and passes standalone fresh compilation. actual_faithful_augmented_productE expands the dedicated structured coefficient-product as the sum of the head, generated HAETAE `poly_dot`, and adjusted error. checked_mode2_parent_m23_finalize_faithful_augmented_paper_as_qj proves coefficientwise congruence of this product with faithful qj for the actual checked parent. The product takes only the actual matrix/secret as input; the middle 2-by-3 block is `poly_dot` reconstructed from the specified columns. The verdict for this work is therefore `GO-KG-FAITHFUL-AUGMENTED`. However, this dedicated mod-$2q$ evaluator is not the existing mod-q security `matrix_vec_mul`, nor is it security-side ExpandVecA equivalence. It therefore does not remove `STOP-KG-AVEC-QJ`.

`ImplementationSecurityGoals.ec` lists these items as `obl_*` operations. Compiling the file means that the goal names and composition interface are well-formed, not that each boolean has been proved true.

6.7 Top-level functional correctness is still a specification

`TopLevelGoals.ec` declares

$$\begin{aligned} \mathsf{FC}_{\mathrm{KG}}: &\quad \mathsf{JKeyGen}_2 = \mathsf{PaperKeyGen}_2, \\ \mathsf{FC}_{\mathrm{Sign}}: &\quad \forall sk, M, \mathsf{JSign}_2(sk, M) = \mathsf{PaperSign}_2(sk, M), \\ \mathsf{FC}_{\mathrm{Verify}}: &\quad \forall vk, M, \sigma, \mathsf{JVerify}_2(vk, M, \sigma) = \mathsf{PaperVerify}_2(vk, M, \sigma) \end{aligned}$$

as boolean specifications. For Sign/KeyGen, equality here includes equality of probability distributions. The current prefix, memory, transcript and codec theorems provide necessary deterministic subsquares, not the full equalities.

6.8 The security composition formula is still a conditional interface

The goal connecting implementation and paper scheme has the form

$$\mathop{\mathrm{Adv}}_{\mathrm{Jasmin}}^{\text{euf-cma}} \leq \mathop{\mathrm{Adv}}_{\mathrm{paper}}^{\text{euf-cma}} + \delta_{\mathrm{KeyGen}} + q_{\mathrm{Sign}}\delta_{\mathrm{Sign}} + \delta_{\mathrm{Verify}} + \delta_{\mathrm{Encoding}}$$

The current file declares loss variables and defines the inequality's interface, but has not discharged every δ and composed them into a final theorem.

The paper-level reduction interface further reduces to MLWE, BST-MSIS, ROM and rejection/fork errors. `SecurityAssumptions.ec` restricts the cryptographic interfaces that may remain in the final result to MLWE, BST-MSIS and ROM. This does not mean that concrete game instances and a parameterized reduction have been connected to the implementation's exact byte-level scheme.

Caution 6.3 (Distinguishing declaration kinds)**.** Operations, assumptions and theorems are different. An operation declared without a body, such as `opobl_sampler_distribution: bool.`, is an abstract symbol expressing a goal. It is neither an axiom assuming the goal true nor a theorem proving it true. The project's verification scripts separately prohibit and scan for authored axioms and proof holes.

6.9 Trust boundary and out-of-scope items

The current evidence chain trusts or fixes the following.

- The 46-page HAETAE specification PDF with a recorded SHA-256 [1];
- the pinned `haetae-ref-jasmin/` implementation and focused extraction [2];
- the EasyCrypt kernel, Jasmin model, invoked SMT provers and build toolchain;
- source hashes and generated extraction/certificate manifests.

The conclusions of this sprint exclude the following.

- Parameter sets other than mode 2;
- physical side channels, faults and RNG quality;
- claims that memory safety or constant-time automatically follow from EUF-CMA;
- end-to-end correctness of all actual hardware/compiler components;
- claims that full KeyGen, Sign, Verify or implementation EUF-CMA are complete.

6.10 Current claim matrix

Claim	Status	Exact boundary
VK prefix of packed SK	**PROVED**	First 992 bytes upon return from the actual generated packer/Internal KeyGen
raw μ generated seam	**PROVED**	64-to-32 prefix on the raw branch and related key/pre/message regions
position-64 challenge suffix	**PROVED**	32-byte absorb from equal initial state/position
actual API key copy/address/frame	**PROVED**	Under each named valid/canonical/disjoint contract
accepted Verify tail/input binding	**PROVED**	Direction accept $\Rightarrow$ tail/hash inputs
stored observed μ equality	**PARTIAL**	Missing product/replay transport
signature prefix inverse	**PROVED**	Canonical challenge/low, first 1056 bytes
HBZ leaf inverse	**PROVED**	First 1024 coefficients, excluding rANS
actual rANS table and pure step	**PROVED**	Concrete 13-symbol table, specified state range
pure normalization byte trace	**PROVED**	Canonical symbol list, not an actual loop
actual array/list and word-step bridge	**PROVED**	1024-symbol suffix recurrence, literal table word equalities
actual final serialization leaf	**PROVED**	Composition of four actual stores with little-endian serialization of the accumulated trace
actual encoder success size	**PROVED**	If it returns, failure or $0 \le off \le 1020$, $4 \le 1024 - off \le 1024$
actual inner normalization	**PROVED**	Byte segment, frame, no-underflow and exact pure 0..2 exit
actual encoder trace refinement	**PROVED**	Complete suffix equality in success-conditioned partial correctness
actual decoder trace refinement	**PROVED**	Exact-trace partial correctness, 1024-symbol equality, exact byte consumption and tail frame
actual rANS harness control	**PROVED**	Only the actual call order and success/failure branches are complete
actual rANS core inverse	**PROVED**	Success-conditioned partial correctness of actual encoder/copy/decoder
actual encoder success witness	**PROVED**	Failure exclusion for returning fixed all-zero/all-6 executions; no termination/losslessness
production full-HBZ wrapper inverse	**PROVED**	Success-conditioned composition of prepare/apply, rANS core and production lift is complete

Claim	Status	Exact boundary
actual KeyGen matrix/finalize snapshot	**PARTIAL**	KG-2-like decomposition and snapshot mod-$2q$ identity proved; in the frozen paper, `STOP-KG-NTT` after the final NTT-word rewrite
post-freeze NTT/Rq/HAETAE row products	**PROVED**	Pure spectral action, native R_q 3-column/4-column row products and KeyGen 3-column HAETAE representation transport; actual Verify procedure excluded
post-freeze actual `avec`/paper a, qj	**PARTIAL**	Deterministic expansion and separate qj snapshot composition proved; `STOP-KG-AVEC-QJ` at the synthetic security semantics mismatch
faithful augmented coefficient-product composition	**PROVED**	Standalone actual-carrier structured product $\equiv qj$; outside the frozen 82-target set; generic security-model `matrix_vec_mul`/ExpandVecA equivalence excluded
actual Sign accepted core	**PARTIAL**	Only call and accept-branch control for the three actual helpers; `STOP-SIGN-CHAL-MODE2`
actual Verify core sequence	**PARTIAL**	Actual five-helper sequence, V-1/V-2/V-5/V-6 word projections, local norm/tail results; frozen `STOP-VERIFY-MATRIX-CRT`
h suffix codec	**DEFERRED**	Scope moved to Week 16 MINCORE KeyGen; inverse not attempted
full signature codec	**BLOCKED**	h, metadata, padding and residual frames remain after the HBZ wrapper
Sign output tail reach	**PARTIAL**	Only prefix and success-conditioned HBZ wrapper closed; h, metadata/padding, norm/hint/arithmetic remain
FC-KeyGen/Sign/Verify	**SPECIFIED**/ **PARTIAL**	Only some deterministic lower-level seams proved, including the KeyGen snapshot
Implementation EUF-CMA	**SPECIFIED**	Loss and paper-reduction connections incomplete

7 Dependency-ordered next steps and contribution points

7.1 The current single priority: repairing security ExpandVecA semantics

The latest post-freeze gate is `STOP-KG-AVEC-QJ`. The first obligation is

> **OBL-KG-SECURITY-EXPANDVECA-SEMANTICS**: a theorem replacing/refining synthetic security ExpandVecA with actual SHAKE128/rejection semantics

Post-freeze NTT/Rq/HAETAE transport and the interpretation of actual `avec` as paper a have compiled. However, the existing security-side object is a synthetic generator, neither the actual SHAKE128/rejection expansion nor paper qj. Composing the full KeyGen equation without repairing this mismatch would equate the concrete coefficients 44985 and 401. Meanwhile, the frozen paper's `STOP-VERIFY-MATRIX-CRT` is a separate status. Although a pure 2-by-4 row-product specialization became available post-freeze, the actual Verify helper loop and CRT/freeze procedure theorems remain open.

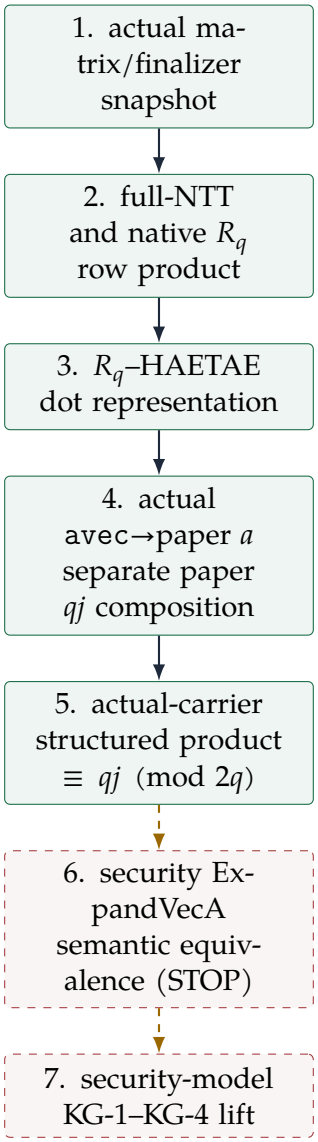


Figure 4: The current post-freeze KeyGen cut. The coefficient-product dedicated to actual carriers is closed, but work stopped at generic security ExpandVecA/`matrix_vec_mul` semantics.

On the codec line, Week 15 closed failure exclusion for returning fixed all-6 executions. Fixed-input termination/losslessness remain open, however, and the h suffix codec became **DEFERRED** when scope moved to this one-week MINCORE effort. Both facts are independent of the current KeyGen blocker.

7.2 Stage 1 complete: encoder

OBL-RANS-ENCODE-REFINEMENT was closed by actual_rans_encode_trace_closure, which opens actual `_rans_encode`, and its expanded corollary actual_rans_encode_trace_refinement.

The success-branch postcondition establishes all of the following together.

- The actual output suffix equals the canonical symbol list's `trace_bytes` pointwise;
- at each outer iteration, the actual word state equals the suffix's pure `encode_trace` state;
- actual off and trace-byte length satisfy $off + |\text{TraceBytes}| = 1024$;
- the already proved final 4-byte state serialization is composed before the normalization suffix, fixing segment order;
- the unwritten-prefix frame is preserved;
- the failure branch is not hidden.

Week 10's array/list suffix recurrence, actual reciprocal word step, inner byte/frame/no-underflow results, final serialization leaf and successful offset/size ranges were connected to Week 11's tail-aware outer/inner invariants, generated table load–protect–word-update lemmas and final-store composition. Consequently, the suffix of the actual returned buffer, rather than an arbitrary valid trace or a fresh witness buffer, agrees with the canonical trace bytes.

This is nevertheless Hoare partial correctness. A success witness showing that an actual execution reaches the success branch, and termination, do not follow from this Week 11 theorem alone. Week 15 separately excluded failure in returning fixed all-6 executions, but termination still remains.

7.3 Stage 2 complete: decoder

OBL-RANS-DECODE-REFINEMENT was closed by actual_rans_decode_trace_refinement, which opens actual `_rans_decode`. Under the exact-trace input relation, induction proved the following.

- At iteration i, the decoded prefix equals the original symbol prefix;
- state/cursor equal the pure x_i, c_i;
- actual table lookup selects the interval partition's unique s_i;
- the nested normalization loop consumes β_i in the exact order;
- $state = 2^{23}$ is derived at the internal loop exit, and the return values establish $off = size$ and $bad = 0$;
- the decoded tail frame.

Connecting encoder and decoder directly in lockstep enlarges the proof surface because of their reverse/forward directions and the intervening buffer copy. The two refinements were verified independently using the already constructed pure trace as a common intermediary. This decoder theorem assumes an input trace and establishes Hoare partial correctness, not termination.

7.4 Stage 3 complete: actual harness inverse

OBL-RANS-CORE-INVERSE was closed by actual_rans_encode_copy_decode_inverse in `Mode2RansCoreActualInverse.ec`. This theorem directly concerns the actual harness defined in `Mode2RansActualInverse.ec` and derives the following semantic conclusions. The two filenames refer respectively to "harness definition/control" and "Week 13 semantic

inverse"; the latter was proved separately to prevent misreading the former's control-only theorem as a semantic result.

- The encoder failure branch remains explicit and does not run the decoder;
- `segment_matches` of the encoder's successful suffix is composed with `slice_eq` of the actual copy;
- the copied buffer supplies the decoder's pointwise exact-read premises;
- actual decoder return gives $bad = 0$, exact consumption, 1024-symbol equality and the tail frame;
- no arbitrary trace, fresh witness buffer or encoder success is added as an external premise.

This is success-conditioned partial correctness. The canonical all-zero array shows non-vacuity of the premises but does not automatically establish existence of encoder success or termination of the complete execution. In Week 13, success reachability was a separate obligation, **OBL-RANS-ACTUAL-SUCCESS-WITNESS**; Week 15 closed only its fixed all-6 Hoare postcondition.

7.5 Stage 4 complete: production full-HBZ wrapper

OBL-SIG-HBZ-ENCODE-DECODE was closed by signature_pack_unpack_hbz_full_actual_exact and signature_pack_unpack_hbz_full_inverse_mode2. The chain directly opening actual `_encode_hb_z1_full` and `_decode_hb_z1_full` concludes the following.

- Actual prepare maps canonical HBZ coefficients to a canonical symbol stream;
- the actual rANS core theorem's failure/success branches are preserved;
- on success, actual apply maps recovered symbols back to the original coefficients;
- wrapper-level size, state and input/output tail frames are connected;
- no actual success witness or decoder result is required again as an external premise.

This stage is **PROVED** as success-conditioned partial correctness [19, 9]. General reachability of the actual success branch and termination are separate theorems. Week 15 excludes failure only for returning executions on the fixed all-zero input.

7.6 Stage 5 complete: fixed all-6 success postcondition

Week 15's actual_rans_encode_all_six_success derives $bad = 0$ for returning fixed-input executions by contradicting the theorem that actual failure requires trace length > 1020 with the all-6 trace budget ≤ 1020. Focused/production lifts establish $4 \leq size \leq 1024$, exact trace/frame and the zero coefficient round trip together [20, 8].

The completion verdict at this stage is fixed-input Hoare partial correctness. It is not promoted to termination, losslessness, non-vacuous execution or `phoare`. The originally planned subsequent h suffix, metadata/padding, full signature inverse and Sign output tail reach remain respectively deferred or blocked after the transition to Week 16 MINCORE.

7.7 Stage 6 frozen stop: KG–NTT–MUL audit

output_row_from_mode2_ntt_words and actual_m23_matrix_snapshot_rows_explicit closed the final sound edge rewriting `KeygenM23ArithmeticSpec.output_row` into actual

full-invNTT/pointwise word expressions and applying them to two active rows [21]. At the frozen-paper point, the following two arrows were absent:

$$\text{array256_mont}(\text{full_invntt}(\hat{a} \odot \text{full_ntt}(p))) \not\Rightarrow (\text{full_invntt}\,\hat{a}) \star_{R_q} p,$$

and the adapter transporting native `Rq.poly` multiplication to the security model's integer-list `poly_mul/matrix_vec_mul` was also absent. The first arrow required odd-root orthogonality and full-NTT convolution. The time-bounded audit did not assume them as new premises and ended with `STOP-KG-NTT`. Post-freeze work closed these pure-algebra and R_q–HAETAE representation gaps, but does not change this historical frozen verdict itself.

7.8 Stage 7 frozen: accepted Sign core

Sequential execution of the following three actual helpers was composed in a transparent harness and compiled through its control theorem.

`_sf_round_challenge_mode2` `_sf_z_check` `_sf_hint_mode2`

The compiled control theorem in `Mode2SignAcceptedCore.ec` is

actual_sign_accepted_core_branch_control_mode2

However, the headline S-1–S-7 theorem was frozen at `STOP-SIGN-CHAL-MODE2` because `sf_challenge_mode2_highbits_lsb_sampleinball_correct` is absent. For returning, accepted executions, commitment/challenge, the response $z = y + (-1)^b cs$, two exact integer norm predicates and the coefficientwise hint equation must be derived [23, 3].

The headline theorem must not yet be counted as a compiled proof. Its precondition must not assume `reject=0` or the final equations. Sampler distributions, signing-loop termination, packing and the complete 1474-byte signature are outside this stage's scope.

7.9 Stage 8 frozen stop: actual Verify helper chain

In `Mode2VerifyCoreSequence.ec`, actual_verify_core_sequence_branch_control_mode2 and actual_verify_core_sequence_input_snapshots_mode2 fix the actual Verify five-helper call sequence and input snapshots. `Mode2VerifyPrepareNorm.ec`, `Mode2VerifyRecover.ec`, and `Mode2VerifyTailChallenge.ec` respectively provide verify_prepare_z1_wprime_mode2_word_exact, actual_sign_verify_recover_w_z2_mode2_word_semantics, verify_tail_exact_trace_mode2, and poly_mismatch_mode2_word_exact [25]. Thus V-1/V-2/V-5/V-6 projections, the W64 norm gate, and tail trace/mismatch expression are closed at helper-local scope.

However, `verify_matrix_ntt_acc_mode2_cols4_correct` for actual 2-by-4 `_polyvec_ntt/_polymat_pointwise_acc/ _polyvec_invntt`, and `verify_crt_freeze_mode2_word_exact` for actual from-CRT/freeze, are absent. The challenge leaf `verify_tail_m23_highbits_lsb_sampleinball_correct` also remains independently open. This is the frozen `STOP-VERIFY-MATRIX-CRT`. The post-freeze pure verify_mode2_row_product_from_reprs supplies the first procedure leaf's algebraic goal but does not replace actual loop refinement.

7.10 Stage 9 in progress: post-freeze actual avec/qj audit

rq_mul_coeff_foldr_to_bigi, full-NTT spectral action, generic row products, native R_q 3-term products and HAETAE dot representation are now **PROVED**. The actual sampler side also

interprets nonces 515/516, SHAKE128 framing, little-endian parsing, < 64513 rejection and unchanged storage as paper a, and separately composes paper qj into the snapshot equation [12, 10].

The first current goal is **OBL-KG-SECURITY-EXPANDVECA-SEMANTICS**. The specification must be replaced/refined so that security `public_rounding_vector_seed` has the same semantics as actual expansion, and a coefficientwise actual-array representation must be proved. The actual-carrier width-6 coefficient-product packaging that separately preserves the paper qj correction is closed in the latest `KgFaithfulAugmentedModel.ec`. Through actual matrix/secret concretization, generated HAETAE `poly_dot` transport and actual_faithful_augmented_productE, this file composes the actual checked parent's product equation $\equiv qj \pmod{2q}$ as a standalone, freshly compiled theorem. This product takes only the actual matrix/secret as input and reconstructs the middle 2-by-3 block from its columns, giving the verdict `GO-KG-FAITHFUL-AUGMENTED`. The remaining obligation is a separate correspondence theorem replacing/refining the synthetic security object with actual ExpandVecA semantics without incorrectly identifying these dedicated mod-$2q$ block semantics with the existing mod-q, width-4 security model.

When closing each parent, check that the child theorem's premises follow from the actual preceding procedure's conclusions. Requiring the desired equality again as an external precondition is a conditional restatement, not composition.

7.11 Treat the frozen product/replay line separately

Independently of the codec line, **OBL-MU-PRODUCT-REPLAY** remains an explicit paper boundary. Resuming it requires one of the following, not another wrapper.

- A sound relational sequencing lemma preserving hash return values until they are stored in ghost observations;
- a product program handling both continuations and its semantics;
- justified one-sided elimination after first proving losslessness/termination of the Sign continuation;
- a minimal instrumentation change making the trace preserve the hash-time snapshot, with a renewed proof of actual-to-trace exactness.

None of these is a simple byte lemma. Program-logic design and theorem-surface trade-offs must be considered first.

7.12 Problems to which pure algebraists can contribute directly

Participation question 7.1 (Packaging rANS as a restricted bijection)**.** Using the sets

$$\mathcal{X}_s = \{x : 2^{23} \le x < 2^{31}\}, \qquad \mathcal{N}_s = \{r : 1 \le r < x_{\max}(s)\}$$

and 0/1/2-byte fibers, can normalization–fast-step be packaged as an explicit partial bijection? The goal is to combine the current individual lemmas into one induction theorem that is easy to reuse in actual loop invariants.

Participation question 7.2 (Representation quotients and canonical sections)**.** Can a common structure describe, for each signature component, the raw word space, byte quotient, canonical system of representatives and decoder section at once? A new abstraction should reuse the actual existing predicates and remove only duplication across prefix/HBZ/future h codecs.

Participation question 7.3 (ExpandVecA, R_q, and mod-$2q$ representations)**.** How can actual SHAKE128/rejection `avec` be represented as a faithful security matrix while preserving the exact propositions and premises of the closed NTT, R_q, and HAETAE results? Without mixing the paper qj correction into a modulo-q vector, how can the new actual-carrier structured coefficient-product be connected to generic ring `matrix_vec_mul` defined solely from the augmented matrix/secret and to the security model? The Verify side needs a separate commutative square connecting the same pure 4-column row product to actual NTT/CRT/freeze procedure theorems.

Participation question 7.4 (Integer bounds in high/low decomposition)**.** Can the canonical-representative conditions between the paper equation $r = \alpha\,\mathrm{HighBits}(r) + \mathrm{LowBits}(r)$ and actual fixed-width rounding/sign-extension/packing be separated precisely? This is the algebraic core of complete challenge-input and norm/hint correctness.

Participation question 7.5 (Rejection distribution)**.** Can a retry loop modeled as a partial map be lifted to a probability kernel, and its accepted distribution and termination/loss bound connected to the paper sampler? The interface must keep deterministic Hoare theorems distinct from distribution theorems.

7.13 Where algebraic-geometric skills are useful

Although the current work is not a scheme-theoretic proof, the following habits are directly useful.

- Local-to-global reasoning that compares only the necessary restrictions rather than entire objects;
- making transition maps and compatibility diagrams between different representation charts explicit;
- separating quotients from representative sections;
- checking with witnesses that fibers are nonempty and examining non-vacuity of conditional theorems;
- decomposing a large goal into commutative squares and obstructions.

These habits must not be turned into a claim that "the current code formalizes a sheaf." The actual proof terms concern arrays, words, memory and procedure semantics.

7.14 Checklist for submitting a new theorem

1. Do its name and comments state the actual/generated/pure boundary?
2. Does it avoid assuming the parent conclusion again as a premise?
3. Does it distinguish termination, partial correctness and losslessness?
4. Are all canonicality, address-range, no-wrap and separation premises visible?
5. Does it identify whether return values, global state or ghost observations are being compared?
6. Is a frame needed for unused array or memory regions?
7. Is it non-vacuous through a concrete witness to its premises or an actual predecessor?
8. Is it free of authored axioms, admit/sorry/abort and debug declarations?

9. Is it included in the manifest and freshly compiled with `-no-eco`?
10. Have parent statuses in the claim ledger and theorem graph been updated without overstatement?

Passing this checklist is what turns "mathematically plausible" into "verified for the current implementation."

A Theorem and source index

This appendix maps the mathematical statements in the main text back to actual repository declarations. Because line numbers change during editing, it uses file paths and theorem identifiers. Always determine the final verdict by reading the premises of the relevant EasyCrypt declaration together with the latest `CLAIM_LEDGER.md`.

A.1 Status and scope documents

Path	Role
`PROJECT_SCOPE.md`	mode 2 scope, fixed specification, trust boundary, items not claimed complete
`CLAIM_LEDGER.md`	Operational source of truth by claim ID: status, actual Jasmin target, EasyCrypt evidence, remaining premises
`THEOREM_GRAPH.md`	Dependency DAG from top-level security goals to leaf theorems
`ASSUMPTIONS.md`	Final assumptions, implementation contracts, non-vacuity and policy against overstatement
`PRODUCT_REPLAY_DECISION.md`	Semantic analysis and freeze decision for the stored μ product/replay gap
`SIGNATURE_CODEC_OBLIGATIONS.md`	mode 2 signature layout and staged codec obligations
`RANS_PROOF_DESIGN.md`	rANS proof decomposition and design choices
`RANS_BYTE_STACK_INVARIANT.md`	The *xs*/*cuts*/*bytes* invariant connecting the reverse encoder and forward decoder
`WEEK9_REPORT.md`	Historical Week 9 baseline of the pure byte stack and actual control boundary
`RANS_ENCODER_INVARIANT.md`	Actual encoder outer-loop invariant, Week 10 design boundary and Week 11 actual trace closure
`WEEK10_REPORT.md`	Historical `CONTINUE-ENCODER` verdict and 50-target verification results
`WEEK11_REPORT.md`	Historical `GO-ENCODER` verdict, 57-target verification results and actual encoder trace closure
`RANS_DECODER_INVARIANT.md`	Actual decoder's pure state/cursor/segment coordinates, outer/inner invariants and exact-trace boundary
`WEEK12_REPORT.md`	Historical `GO-DECODER` verdict, 65-target verification results and actual decoder refinement

Path	Role
RANS_CORE_COMPOSITION.md	Week 13 transport structure between encoder segments, actual copy slices, decoder pointwise reads and configured states
MODE2_HBZ_CODEC_OBLIGATIONS.md	Latest layer-by-layer status of HBZ leaf, rANS core and production-wrapper obligations
WEEK13_REPORT.md	Historical GO-CORE verdict, 67-target verification results and actual core inverse
HBZ_FULL_WRAPPER_COMPOSITION.md	Failure/success contracts at the focused full-HBZ harness and production SignaturePack/Unpack boundary
WEEK14_REPORT.md	Historical GO-HBZ verdict, 72-target verification results and production full-HBZ wrapper closure
RANS_ACTUAL_SUCCESS_WITNESS.md	Actual prepare's all-6 stream, 1020-byte budget, failure-cause contradiction and production zero round trip
WEEK15_REPORT.md	Historical GO-WITNESS verdict, 74-target verification results and fixed all-6 Hoare success; termination/losslessness excluded
WEEK16_MINCORE_PLAN.md	KeyGen/Sign/Verify plan for one-week MINCORE and frozen stopping criteria for KeyGen/Sign/Verify
WEEK16_KG_REPORT.md	Historical first-task CONTINUE-KG verdict and 76-target pre-change baseline
WEEK16_KG_NTT_MUL_REPORT.md	Final sound output_row rewrite, exact missing convolution/adapter leaves, STOP-KG-NTT and subsequent Sign boundary
WEEK16_SIGN_REPORT.md	Direct calls/control of three actual Sign helpers, missing full challenge semantics and STOP-SIGN-CHAL-MODE2
WEEK16_VERIFY_REPORT.md	Sequence of five actual Verify helpers and helper-local word/control results, 82-target frozen verdict STOP-VERIFY-MATRIX-CRT
manifests/paper-artifacts.md	Scope of frozen paper/PDF/82-target evidence and the boundary excluding post-freeze work
POST_FREEZE_KG_RQ_HAETAE_REPORT.md	Full-NTT/native R_q row products and HAETAE representation transport; remaining actual avec/security semantic boundary
POST_FREEZE_KG_ACTUAL_AVEC_QJ_REPORT.md	Actual SHAKE128/rejection avec and paper a, qj, concrete mismatch with the synthetic security object, STOP-KG-AVEC-QJ
POST_FREEZE_KG_FAITHFUL_AUGMENTED_MODEL_REPORT.md	Explicit 2-by-6 actual carrier, matrix/secret-only block product and GO-KG-FAITHFUL-AUGMENTED; security-model correspondence excluded

A.2 Packed keys and memory bridges

File	Main declarations	Mathematical role
easycrypt/refinement/keygen/PackedKeyPrefix.ec	vk_prefix_eq_set_after; copied_prefix_step; copied_prefix_complete	Pure array invariant for prefix equivalence
easycrypt/refinement/keygen/ExtractedPackedKeyPrefix.ec	pack_vec_eta_to_prefix_frame; pack_vec2_eta_to_prefix_frame; pack_sk_m23_mode2_vk_prefix	Actual packer prefix and frame for subsequent writes
Same file	keypair_full_m23_mode2_return_prefix; keypair_internal_mode2_return_prefix	Lift through generated internal KeyGen return
easycrypt/support/Mode2KeyMemoryBridge.ec	keygen_prefix_reaches_mu_memory	Translate a local-array prefix into a global-memory load relation
easycrypt/refinement/composition/ApiKeyMemoryBridge.ec	keygen_export_vk_mode2_prefix; keygen_export_sk_mode2_prefix; sign_import_mode2_sk_prefix; verify_import_mode2_vk_prefix	Export/import contracts of actual copy helpers
Same file	keygen_export_sk_mode2_prefix_frames_vk; sign_verify_api_importer_same_inputs	Disjoint frames and connection of Sign/Verify inputs
easycrypt/refinement/composition/RawApiAddressBridge.ec	canonical_ui64_address_roundtrip; mode2_vk_region_bridge; mode2_sk_region_bridge; mode2_sig_region_bridge	Canonical integer/$W64$ address correspondence
easycrypt/refinement/composition/RawApiKeygenSequentialExport.ec	keypair_raw_api_trace_exports_matching_prefixes; keypair_raw_api_exports_matching_prefixes	Sequential external exports by the actual raw KeyGen caller

A.3 Frozen Week 16 MINCORE KeyGen, Sign and Verify boundaries

The following 8 files in the frozen 82-target manifest cover actual KeyGen matrix/finalizer, Sign control and Verify helper-local semantics. The essential boundary in this table is that partial results must not be renamed as complete headline results.

File	Main declarations	Mathematical role
easycrypt/refinement/keygen/Mode2KeygenSnapshotAlgebra.ec	snapshot_residue_exact_low_high; raw_sum_raw_residue_congruent_mod_2q; snapshot_expression_from_product_congruent_mod_2q	Canonical-residue $r = 2b_1 + b_0$, low/high bounds and doubled mod-$2q$ congruence
easycrypt/refinement/keygen/Mode2KeygenCoreEquation.ec	actual_m23_matrix_finalize_snapshot; finalize_semantic_output_low_high_decomposition; finalize_semantic_output_snapshot_mod2q_zero	Actual `_kp_m23_matrix(2,3)` snapshot and exact/semantic results of actual `_keypair_finalize_m23(512)`
Same file	actual_m23_matrix_finalize_semantic_snapshot	Strongest transparent-harness theorem for representation, finalizer semantics, KG-2-like decomposition, adjusted s_2, snapshot-only congruence and tail frame

File	Main declarations	Mathematical role
easycrypt/refinement/keygen/Mode2KeygenNttMulBridge.ec	output_row_from_mode2_ntt_words; output_row_repr_from_mode2_ntt_words; actual_m23_matrix_snapshot_rows_explicit; keygen_mode2_row_product; mode2_row_product_haetae_dot; actual_m23_matrix_snapshot_rows_row_product	Final full-invNTT/pointwise-word representation rewrite of the actual NTT chain and consequences for two active rows of the direct matrix/finalizer harness. The last three theorems compose post-freeze shared NTT/Rq/HAETAE results for KeyGen's 3 columns
Same file	Absence, at the frozen point, of odd-root orthogonality/full-NTT convolution and Rq.poly representation transport	Historical boundary where frozen **OBL-MINCORE-KEYGEN** was STOP-KG-NTT; those pure/representation leaves were closed post-freeze
easycrypt/refinement/sign/Mode2SignAcceptedCore.ec	actual_sign_accepted_core_branch_control_mode2	Direct actual round-challenge, z-check and accepted-only hint calls; S-1–S-7 unproved at STOP-SIGN-CHAL-MODE2
easycrypt/refinement/verify/Mode2VerifyCoreSequence.ec	actual_verify_core_sequence_branch_control_mode2; actual_verify_core_sequence_input_snapshots_mode2	Call/branch sequence and input snapshots of five actual helpers
easycrypt/refinement/verify/Mode2VerifyPrepareNorm.ec	verify_prepare_z1_wprime_mode2_word_exact	Helper-local semantics of V-1/V-2 word projections and the W64 norm gate
easycrypt/refinement/verify/Mode2VerifyRecover.ec	actual_sign_verify_recover_w_z2_mode2_word_semantics	V-5/V-6 word semantics of the recover path
easycrypt/refinement/verify/Mode2VerifyTailChallenge.ec	verify_tail_exact_trace_mode2; poly_mismatch_mode2_word_exact	Tail trace and mismatch word expression; actual matrix/CRT/challenge leaves unproved at STOP-VERIFY-MATRIX-CRT

A.4 Post-freeze shared NTT/Rq and actual avec/qj

The following files form a separate proof surface that does not increase the frozen 82-target count. Shared NTT theory supplies pure mathematics. The three KgActualAvecQj*.ec theories in post-freeze/ record the actual sampler/finalizer boundary and discovered mismatch as compiled theorems; the latest KgFaithfulAugmentedModel.ec closes faithful augmented coefficient composition as a standalone theorem.

File	Main declarations	Mathematical role
haetae-ntt-verify/easycrypt/NTTRowProductSpec.ec	coefficient_row_product; matrix_inverse_repr; vector_forward_repr; freeze_repr_preserves_poly	Generic row-product and NTT/freeze representation contracts
haetae-ntt-verify/easycrypt/NTTFullSpectralAction.ec	rq_mul_coeff_foldr_to_bigi; full_ntt_montgomery_spectral_action; full_ntt_montgomery_row_product	Negacyclic coefficient sum, full-NTT convolution and generic row products

File	Main declarations	Mathematical role
haetae-ntt-verify/easycrypt/NTTMode2RowSpecializations.ec	keygen_mode2_row_product_from_reprs; verify_mode2_row_product_from_reprs	Pure Mode 2 specializations for KeyGen 3 columns/Verify 4 columns; actual Verify loop excluded
haetae-ntt-verify/easycrypt/RqHAETAEBridge.ec	rq_poly_add_repr; rq_poly_mul_repr; rq_poly_dot3_repr	Transport from native R_q operations to HAETAE coefficient/list representations
post-freeze/KgActualAvecQjSemantics.ec	mode2_uniform_stream_paper_a_flat; paper_qj_row0_coeff0; paper_qj_remaining_row	Interpret actual SHAKE128/rejection avec as paper a, and coefficient semantics of $qj = q(1,0)^T$
post-freeze/KgActualAvecQjComposition.ec	mode2_sampler_facts_actual_paper_a; actual_snapshot_zero_adds_paper_qj; checked_mode2_parent_m23_finalize_actual_avec_qj	Partial composition of actual sampler, matrix/finalizer snapshot and separate paper qj term
post-freeze/KgActualAvecQjBlocker.ec	security_named_qj_mode2_row0_coeff0; security_named_qj_is_not_paper_qj	Compiled mismatch between synthetic security coefficient 401 and paper $qj[0,0] = 64513$
post-freeze/KgFaithfulAugmentedModel.ec	faithful_augmented_matrix_coeff; faithful_augmented_secret_coeff; actual_faithful_augmented_matrix; actual_faithful_augmented_secret; faithful_middle_matrix_poly; faithful_middle_secret_poly; faithful_augmented_matrix_vector_product_coeff; actual_faithful_augmented_productE; faithful_generated_row_product_haetae_dot; faithful_augmented_product_qj_from_generated_dot; mode2_m23_facts_generated_row_product; checked_mode2_parent_m23_finalize_faithful_augmented_paper_as_qj	Actual width-6 carrier, generated poly_dot transport and paper-shaped mod-2q composition of the structured coefficient-product; standalone fresh compilation; product consumes only explicit matrix/secret; GO-KG-FAITHFUL-AUGMENTED; generic security-model matrix_vec_mul/ExpandVecA equivalence excluded
post-freeze/check-kg-actual-avec-qj-trace.py	zero-seed row 0 coefficient 0: actual 44985, synthetic 401	Independent deterministic FIPS-202 trace witness

A.5 μ, challenge, actual API trace

File	Main declarations	Mathematical role
easycrypt/refinement/composition/ExtractedMuHashPrefix.ec	sign_sk_verify_vk_absorb_mode2; sign_verify_final_squeeze_mu32_prefix; sign_verify_raw_mu_top_wrappers_prefix	Key/hash/squeeze relations of the actual generated helper chain
easycrypt/refinement/composition/ExactMuTopControl.ec	raw_prelen_shr63_zero; raw_prelen_branch_guard	Raw-path selection in the actual Verify branch
Same file	sf_mu_rawpre_refines_raw_top; verify_hash_mu_raw_refines_raw_top; sign_verify_generated_raw_mu_prefix	64-to-32 μ relation of actual generated top procedures

File	Main declarations	Mathematical role
easycrypt/refinement/composition/ExactMode2RawMuComposition.ec	generated_raw_mu_preconditions_from_keygen_prefix; generated_raw_mu_to_challenge_suffix_zero_loss	From the KeyGen prefix to actual position-64 challenge absorb
easycrypt/refinement/composition/RegionLocalMuEquivalence.ec	region_eq_head; region_eq_tail; sign_verify_raw_mu_top_wrappers_prefix_regionwise	Weaken whole-memory equality to read-region equality
easycrypt/refinement/composition/RegionLocalMuTop.ec	sign_verify_generated_raw_mu_prefix_regionwise	Lift the region-local relation to actual generated top procedures
easycrypt/refinement/composition/RawApiSignOutputFrame.ec	sign_output_copy_exact_and_frames_reused; sign_raw_api_frames_reused_regions	Frame for actual Sign output and siglen store
easycrypt/refinement/composition/RawApiVerifyAcceptTrace.ec	verify_cryptolab_actual_accept_implies_trace_tail_reached; verify_cryptolab_actual_accept_binds_hash_inputs	Accept ⇒ tail and actual hash-input binding
easycrypt/refinement/composition/RawApiDirectObservedMu.ec	raw_sign_then_verify_actual_exact_trace; sign_then_verify_trace_accept_implies_tail_reached; sign_then_verify_accept_binds_observed_inputs	Exactness between actual sequential Sign–Verify and the ghost trace
Same file	No direct stored `observed_mu` relation	Exact point where **OBL-DIRECT-OBSERVED-MU** is **PARTIAL**

A.6 Signature prefix and HBZ leaf

File	Main declarations	Mathematical role
easycrypt/refinement/sign/Mode2SignaturePrefixCodec.ec	canonical_challenge; canonical_signed_low; prefix_codec_preconditions_satisfiable	Canonical-representative conditions and non-vacuity
easycrypt/refinement/sign/Mode2SignaturePrefixPack.ec	pack_sig_prefix_mode2_layout	Challenge/low byte layout of the actual pack loop
easycrypt/refinement/sign/Mode2SignaturePrefixUnpack.ec	unpack_sig_prefix_mode2_layout	Decode layout and tail frame of the actual unpack loop
easycrypt/refinement/sign/Mode2SignaturePrefixRoundTrip.ec	pack_unpack_sig_prefix_mode2_roundtrip	Actual decode-after-encode on a canonical prefix
easycrypt/refinement/sign/Mode2HbzCodecSpec.ec	canonical_hbz_mode2; prepared_hbz_prefix; decoded_hbz_prefix	HBZ canonical domain and coefficient/symbol relation
easycrypt/refinement/sign/Mode2HbzPrepare.ec	encode_hb_z1_prepare_mode2_correct	actual $h_i \mapsto h_i + 6$ leaf
easycrypt/refinement/sign/Mode2HbzApply.ec	decode_hb_z1_apply_mode2_correct	actual $s_i \mapsto s_i - 6$ leaf
easycrypt/refinement/sign/Mode2HbzLeafRoundTrip.ec	encode_prepare_decode_apply_mode2_inverse	Actual leaf inverse and frame for the first 1024 coefficients
easycrypt/refinement/sign/Mode2HbzActualBoundary.ec	pack_target_encode_hb_z1_full_exact_focused; unpack_target_decode_hb_z1_full_exact_focused	Actual procedure identity between focused and full signature extractions

A.7 rANS tables, pure traces and actual boundaries

File	Main declarations	Mathematical role
`easycrypt/refinement/sign/Mode2HbzTableCertificate.ec`	hbz_frequency_positive; hbz_frequency_sum; hbz_intervals_cover_scale; mode2_hbz_pack_unpack_tables_identical	13-symbol partition and pack/unpack table compatibility
Same file	hbz_reciprocal_exact; hbz_fast_step_matches_math	Integer meaning of the actual reciprocal fast quotient
`easycrypt/refinement/sign/Mode2HbzSymbolWordsGenerated.ec`	actual_mode2_hbz_tables_certified	Generated literal arrays satisfy the certificate
`easycrypt/refinement/sign/Mode2RansCore.ec`	pure_rans_step_inverse; hbz_fast_step_decode_inverse; hbz_fast_step_preconditions_satisfiable	Pure/fast single-step inverse and witness
`easycrypt/refinement/sign/Mode2RansNormalization.ec`	encoder_shift8_uint; encoder_low_byte_uint; decoder_append_encoder_byte; encoder_two_byte_decoder_order	Actual $W32/W64$ normalization leaves and byte order
`easycrypt/refinement/sign/Mode2RansByteStack.ec`	renorm_bytes_readback; serialize32_parse_inverse; encode_trace_state_bounds; trace_head_normalization_readback; valid_rans_trace_canonical	Canonical $xs/cuts/bytes$ trace and inductive readback
`easycrypt/refinement/sign/Mode2RansSuffixCopy.ec`	copy_encoded_suffix_correct	Actual suffix slice copy and output tail frame
`easycrypt/refinement/sign/Mode2RansEncodeRefinement.ec`	actual_rans_encode_mode2_control	Historical boundary of actual table/count/*bad* control; Week 11 parent closure is in the table below
`easycrypt/refinement/sign/Mode2RansDecodeRefinement.ec`	actual_rans_decode_mode2_control; mode2_decode_state_satisfiable	Historical boundary of actual decode control and a limited state witness; Week 12 parent theorem is in the table below
`easycrypt/refinement/sign/Mode2RansActualInverse.ec`	actual_rans_harness_branches_on_encoder_result; `actual_core_success_result`	Control boundary of three actual calls; Week 13 semantic inverse is in the table below

Week 10 actual encoder targets. All 6 files below are in the authored target manifest and compile freshly with `-no-eco`. Status refers to each named proposition's scope. The parent encoder trace refinement did not yet exist in Week 10; the following Week 11 table composes these local results through the actual procedure.

File	Main declarations	Mathematical role
`easycrypt/refinement/sign/Mode2RansArrayListBridge.ec`	symbol_suffix_cons; segment_matches_prepend_set; encode_trace_suffix_extension	Array/list suffix, segment match, prefix frame and one-symbol trace recurrence
`easycrypt/refinement/sign/Mode2RansEncoderWordStep.ec`	actual_mode2_encoder_word_step_correct; encoder_word_step_preconditions_satisfiable	Equality between actual word expressions using literal reciprocal/shift/bias/complement and the pure fast step
`easycrypt/refinement/sign/Mode2RansEncoderInnerProgress.ec`	normalization_len_cases; inner_progress_segment_step	Pure 0/1/2 normalization state/cursor/byte-segment progress

File	Main declarations	Mathematical role
easycrypt/refinement/sign/Mode2RansEncoderSerialization.ec	actual_w32_le_bytes_serialize; actual_encoder_final_state_serialization; actual_store_w32_le_prefix_frame	Leaf propositions for four final $W32$ little-endian stores and their frame
easycrypt/refinement/sign/Mode2RansEncoderTrace.ec	encoder_inner_phase_step; encoder_inner_segment_step	Finite phases, byte prepending and segment/frame preservation reused in the actual inner loop
easycrypt/refinement/sign/Mode2RansEncoderActualInner.ec	actual_rans_encode_inner_no_underflow; actual_rans_encode_success_size_bound	Actual nested-loop invariant of generated `_rans_encode` and partial correctness of successful offset/size

Week 11 actual encoder closure targets. These 7 files are also in the authored target manifest and compile freshly with `-no-eco`. They combine the preceding table's transport lemmas into tail-aware invariants, generated word updates, final serialization and the success postcondition of actual `_rans_encode`.

File	Main declarations	Mathematical role
easycrypt/refinement/sign/Mode2RansEncoderGeneratedWordStep.ec	generated_loaded_nested_word_update; generated_outer_word_update_matches	Lemmas connecting generated table loads and nested word expressions to the pure fast step
easycrypt/refinement/sign/Mode2RansEncoderSerializationComposition.ec	actual_store_w32_le_prepend_trace; actual_store_w32_le_prepend_trace_bytes	Compose the four final-state bytes before the normalization trace
easycrypt/refinement/sign/Mode2RansEncoderTailInvariant.ec	encoder_inner_tail_exit_exact; encoder_outer_tail_advance_success	Tail-aware invariant preserving 0/1/2-byte inner exits and one-symbol outer transitions
easycrypt/refinement/sign/Mode2RansEncoderFinalization.ec	encoder_outer_tail_finalize_success	Construct the final-store trace at the outer-iteration exit state
easycrypt/refinement/sign/Mode2RansEncoderGeneratedFinalization.ec	generated_encoder_outer_finalize_success	Connect generated offset calculations and final stores to the pure-trace conclusion
easycrypt/refinement/sign/Mode2RansEncoderActualTraceClosure.ec	encoder_generated_success_outer_post; actual_rans_encode_trace_closure	Exact equality between the successful suffix of actual `_rans_encode` and the canonical trace
easycrypt/refinement/sign/Mode2RansEncoderOuterRefinement.ec	actual_rans_encode_trace_refinement	Success-conditioned partial-correctness corollary explicitly presenting successful offset/size/frame

Week 12 actual decoder semantic-refinement targets. These 8 files also compile freshly with `-no-eco` from the authored target manifest. They separate the pure coordinates of the exact trace, concrete table/word arithmetic, 0/1/2-byte replay, actual outer/inner loops and the generated top Hoare theorem into layers.

File	Main declarations	Mathematical role
easycrypt/refinement/sign/Mode2RansDecoderCursor.ec	`decoder_state_at`; `decoder_cursor`; decoder_parse32_from_trace	Pure trace state/cursor/segment coordinates and initial LE32 parse

File	Main declarations	Mathematical role
easycrypt/refinement/sign/Mode2RansDecoderWordStep.ec	actual_mode2_decoder_lookup_symbol; actual_mode2_decoder_step_correct	Concrete symbol_words lookup and mathematical decoder step
easycrypt/refinement/sign/Mode2RansDecoderNormalization.ec	decoder_replay_complete; decoder_replay_guard_exact; decoder_replay_word_step	Exact 0/1/2-byte replay and guard for the normalization segment
easycrypt/refinement/sign/Mode2RansDecoderActualWord.ec	actual_mode2_decoder_word_update_correct	Equality between the actual $W32/W64$ word update and hbz_math_decode_step
easycrypt/refinement/sign/Mode2RansDecoderGeneratedStep.ec	generated_decoder_symbol_expected; generated_decoder_word_update_matches	Expected-symbol selection by generated table loads and word-step transport
easycrypt/refinement/sign/Mode2RansDecoderCursorSteps.ec	decoder_cursor_step; decoder_cursor_final	One-step recurrence of pure trace cuts and exact final consumption
easycrypt/refinement/sign/Mode2RansDecoderActualTrace.ec	actual_mode2_decoder_trace_input; decoder_outer_trace_live; decoder_outer_trace_exit_components	Actual outer/inner invariants, $x = 2^{23}$, exact cursor and output frame
easycrypt/refinement/sign/Mode2RansDecoderTopHoare.ec	actual_rans_decode_trace_refinement	Exact-trace partial correctness of actual generated _rans_decode

Week 13 actual core composition targets. These 2 files compose the Week 11 encoder theorem, Week 12 decoder theorem and actual copy theorem in one unary harness and compile freshly with -no-eco.

File	Main declarations	Mathematical role
easycrypt/refinement/sign/Mode2RansCoreCompositionBridge.ec	copied_suffix_is_exact_trace; trace_bytes_trace_segment_nth; segment_matches_implies_exact_decoder_segment_input; segment_matches_implies_decoder_word_reads	Non-circular bridge lemmas transporting the encoder suffix and actual copy to global-trace, local-segment and decoder pointwise/W64-read premises
Same file	actual_decoder_input_from_copied_trace; configured_decoder_state_fields; actual_decoder_input_from_configured_trace; encoder_success_size_word_bridge; core_composition_preconditions_satisfiable	Combine copied trace, configured state and no-wrap size in the Week 12 decoder input predicate; canonical all-zero input checks only consistency of the semantic precondition
easycrypt/refinement/sign/Mode2RansCoreActualInverse.ec	actual_rans_encode_copy_decode_inverse	Success-conditioned partial correctness establishing the failure/success disjunction and 1024-symbol round trip of three actual generated procedures in one harness

Week 14 production full-HBZ boundary targets. These 5 files compose the Week 13 core inverse with actual prepare/apply and lift from the focused full-HBZ harness to the production SignaturePack/Unpack harness. All compile freshly with -no-eco.

File	Main declarations	Mathematical role
easycrypt/refinement/sign/Mode2HbzInternalBoundaries.ec	full_encode_prepare_mode2_correct; full_rans_encode_trace_refinement; full_rans_decode_trace_refinement; full_decode_apply_mode2_correct	Connect the focused full wrapper's four internal actual calls exactly to the existing prepare/rANS/apply theorems
easycrypt/refinement/sign/Mode2HbzFullEncodeTrace.ec	actual_encode_hb_z1_full_mode2_trace	Full encoder failure/success branches on canonical HBZ inputs, prepared-symbol trace, size and suffix frame
easycrypt/refinement/sign/Mode2HbzFullDecodeInverse.ec	actual_decode_hb_z1_full_mode2_inverse	From the exact prepared trace, recover $bad = 0$, the original coefficient prefix and tail frame
easycrypt/refinement/sign/Mode2HbzFullActualInverse.ec	actual_hbz_full_encode_decode_inverse_mode2	Failure/success disjunction and success-conditioned coefficient round trip of the focused full-HBZ direct harness
easycrypt/refinement/sign/Mode2HbzSignatureBoundaryLift.ec	signature_pack_unpack_hbz_full_actual_exact; signature_pack_unpack_hbz_full_inverse_mode2	Exact equivalence of the focused harness and production SignaturePack/Unpack harness, and the production Hoare corollary

Week 15 fixed all-6 success targets. These 2 new files and strengthened theorems in the existing closure compose actual all-zero prepare, all-6 capacity and failure causes into the production zero round trip.

File	Main declarations	Mathematical role
easycrypt/refinement/sign/Mode2RansAllSixBudget.ec	actual_prepare_zero_hbz_all_six; symbol6_normalization_len_le1; all_six_normalization_budget; all_six_trace_fits_mode2	Actual prepare's all-6 prefix and conservative 1020-byte normalization/ $[4, 1024]$ total-trace bound
easycrypt/refinement/sign/Mode2RansEncoderActualTraceClosure.ec	actual_rans_encode_failure_trace_cause	Strengthened consequence of the existing target: actual nonzero *bad* requires normalization trace > 1020
easycrypt/refinement/sign/Mode2RansActualSuccessWitness.ec	actual_rans_encode_all_six_success; actual_encode_hb_z1_full_zero_success; signature_pack_hbz_zero_success_mode2; signature_pack_unpack_hbz_zero_success_mode2	Fixed-input failure exclusion, exact trace/frame and zero coefficient round trip from the actual core through focused/production full wrappers

Caution A.1 (Closed fixed-input failure exclusion and remaining termination)**.** The actual encoder suffix, actual suffix copy and actual decoder exact trace were composed in the Week 13 core; Week 14 composed prepare/apply through the production boundary. Week 15 excluded failure in returning actual executions on fixed all-zero/all-6 inputs. These Hoare theorems do not establish fixed-input termination, losslessness, non-vacuous execution or probability-one success. They must not be read as an unconditional total round trip.

A.8 Top-level goals and assumption surface

File	How to read it
`easycrypt/specs/TopLevelGoals.ec`	`FC_KeyGen`, `FC_Sign`, `FC_Verify`, `ImplementationSecurityComposition`, `PaperSecurityReduction` are goal specifications.
`easycrypt/security/ImplementationSecurityGoals.ec`	`obl_*` lists implementation obligations that must not be hidden as assumptions. Compiling the file does not establish the truth of each obligation.
`easycrypt/security/SecurityAssumptions.ec`	Well-formedness of MLWE, BST-MSIS and ROM interfaces and nonnegative bounds that may remain in the final theorem. The concrete reduction connection is separate.

B Glossary of symbols and terminology

Term	Meaning in this document
Proposition	A mathematical statement that can be true or false; here this also includes statements expressing a theorem's premises and conclusion
Theorem	A proposition proved from premises by permitted inference rules; EasyCrypt checks both the named declaration and its proof term
Lemma	An intermediate proposition used to prove a larger theorem
Ring	An algebraic structure with addition and multiplication satisfying the ring axioms
Quotient ring	A ring quotiented by the equivalence relation induced by an ideal; here primarily $R_q = \mathbb{Z}_q[X]/(X^n + 1)$
Module	The analogue of a vector space over a ring; its coefficient scalars may belong to a ring rather than a field
Map	A rule sending elements of one object to elements of another; depending on context, a function or a semantic correspondence induced by a procedure
Isomorphism	A map with an inverse that preserves the relevant structure
Equivalence relation	A relation satisfying reflexivity, symmetry and transitivity
Quotient set	The set of equivalence classes of an equivalence relation
Commutative diagram	A diagram in which different compositions of maps with the same starting and ending points are equal
Partial function	A function defined only on a specified part of its domain, not at every point
Invariant	A proposition preserved across iterations or state transitions
Bijection	A function that is both injective and surjective
mode 2	Fixed parameter set with $n = 256, (k, \ell) = (2, 4), q = 64513, \eta = 1, \tau = 58$
generated procedure	An actual EasyCrypt procedure extracted from pinned Jasmin source using `jasmin2ec`

Term	Meaning in this document
focused extraction	A target narrowly re-extracted for selected calls/tables; procedure identity with full extraction is checked through separate proofs/hashes
pure lemma	A theorem concerning only integer, list and array operations, without calling a generated procedure
leaf theorem	A theorem directly opening one small transformation or word operation within a larger procedure
wrapper/adapter	A procedure or relational theorem connecting an already proved small surface to an actual generated top call
raw/internal	The fixed raw branch and internal mode 2 entry before public context packaging
public/cryptolab caller	The actual API layer including external pointers, lengths, copies and early returns
Hoare partial correctness	The postcondition holds if the procedure returns; termination is separate
pRHL / `equiv`	A probabilistic relational Hoare-logic surface connecting related initial and final states of two programs
exact trace	Equivalence with an instrumented procedure that exposes ghost observations while preserving actual results and final memory
frame	A theorem that array/memory regions outside the target are unchanged across execution
Canonicality	Conditions on word/coefficient representatives chosen so that encode/decode are inverses
prefix	Pointwise equality on the restriction $[0, n)$, not the whole array
region-local	Compares only actual read-footprint regions instead of entire memories
tail reached	A ghost control fact that Verify passed its early rejections and reached the actual tail/hash/challenge path
`bad`	Actual codec success/failure word; usually 0 denotes success and 1 failure, but each theorem's branch conditions must be checked
non-vacuity	Evidence that a concrete state/execution satisfying the premises or success branch actually exists
zero-loss	The deterministic/statistical loss of the specified local relational seam is 0; this does not mean that the entire higher-level δ is 0
`-no-eco`	Verification option compiling an authored target freshly without EasyCrypt cache reuse

C Reproduction and reading paths

C.1 Building only this guide

To build the English edition, run the following in this archived guide's directory, using XeLaTeX and the fonts listed in its README:

```
latexmk -xelatex -interaction=nonstopmode -halt-on-error main.tex
```

This builds the translated PDF only. The historical scripts below require the complete earlier tree restored as described in the archive README. From that restored repository root:

```
sh haetae-topdown-easycrypt/algebraist-guide/build.sh
```

On success, `haetae-topdown-easycrypt/algebraist-guide/main.pdf` is generated. The script checks the existence of source-of-truth files, undefined references/citations and agreement between the 82 snapshot and the proof-target manifest count. Building the PDF does not reverify EasyCrypt theorems.

C.2 The full proof pipeline

From the restored historical repository root:

```
./haetae-topdown-easycrypt/scripts/verify-all.sh
```

The pipeline checks pinned source drift, focused extraction, generated certificate hashes, the authored target manifest, `-no-eco` compilation, proof-hole/axiom/debug scans, selected baselines and the existing research-note LaTeX. The Post-PAPER–FREEZE faithful augmented KeyGen coefficient-product composition record gives 82 authored targets, but actual output at reproduction time always takes precedence.

Verified proposition C.1 (Aggregate result at the time of writing)**.** The frozen complete run recorded in `WEEK16_VERIFY_REPORT.md` and the latest aggregate log has `RESULT PASS`, `authored-targets=82`, `cache=-no-eco`. The historical baseline of 77 was at the KG–NTT stop; one actual Sign-control file and four Verify files were added afterward. The six Week 10 files, seven Week 11 encoder files, eight Week 12 decoder files, two Week 13 core-composition files, five Week 14 full-HBZ boundary files, two Week 15 fixed-input success files, and the Week 16 two KeyGen-snapshot files, one NTT-boundary file, one Sign file and four Verify files are included in manifest-completeness checks, proof-hole/axiom/debug scans and fresh compilation. This PASS denotes compilation and audit of the named targets; it does not promote higher-level **PARTIAL** or **BLOCKED** claims to complete.

Post-freeze shared NTT/Rq and `post-freeze/` theories are not included in these 82 [4]. Shared NTT/Rq and the three `KgActualAvecQj*.ec` files are checked through separate fresh compilation and a deterministic trace checker, without changing the frozen aggregate or paper artifacts [12, 10]. `KgFaithfulAugmentedModel.ec` also passed separate standalone fresh compilation but is not included in the frozen aggregate's 82 targets.

C.3 A 30-minute reading path

1. Read the in/out-of-scope descriptions in `PROJECT_SCOPE.md`.
2. In `CLAIM_LEDGER.md`, compare the ten rows **KG-PREFIX-RETURN**, **OBL-DIRECT-OBSERVED-MU**, **OBL-RANS-ENCODE-REFINEMENT**, **OBL-RANS-DECODE-REFINEMENT**, **OBL-RANS-CORE-INVERSE**, **OBL-SIG-HBZ-ENCODE-DECODE**, **OBL-RANS-ACTUAL-SUCCESS-WITNESS**, **OBL-MINCORE-KEYGEN**, **OBL-MINCORE-SIGN**, **OBL-MINCORE-VERIFY**.
3. Read sections 2, 5 and 6 in the main text.
4. In `WEEK15_REPORT.md` and `Mode2RansActualSuccessWitness.ec`, distinguish fixed all-6 Hoare success from termination/losslessness.
5. Read `WEEK16_KG_REPORT.md` and `Mode2KeygenCoreEquation.ec` for the snapshot-only congruence.

6. In WEEK16_KG_NTT_MUL_REPORT.md and Mode2KeygenNttMulBridge.ec, distinguish the proved NTT-word rewrite from the absent convolution/security-list bridge.

7. In WEEK16_SIGN_REPORT.md and Mode2SignAcceptedCore.ec, inspect actual-call control and the STOP-SIGN-CHAL-MODE2 **PARTIAL** boundary of **OBL-MINCORE-SIGN**.

8. In WEEK16_VERIFY_REPORT.md and the four Mode2Verify*.ec files, distinguish helper-local results from actual matrix/CRT/challenge blockers and check the frozen STOP-VERIFY-MATRIX-CRT.

9. In the three POST_FREEZE_KG_*_REPORT.md reports and three compiled post-freeze/KgActualAvecQj*.ec files, distinguish the closed NTT/Rq/actual avec arrows from the current STOP-KG-AVEC-QJ. Then, in KgFaithfulAugmentedModel.ec, distinguish structured coefficient-product composition consuming only explicit matrix/secret from the absence of generic security-model equivalence.

C.4 A half-day path before contributing proofs

1. RANS_BYTE_STACK_INVARIANT.md
   Map this to the pure trace in Mode2RansByteStack.ec.

2. RANS_ENCODER_INVARIANT.md
   Read the terms of the tail-aware outer/inner-loop invariant.

3. Mode2RansEncoderActualTraceClosure.ec
   Compare the loop invariant with the success/failure branches of the closure theorem that opens the actual procedure.

4. Mode2RansEncoderOuterRefinement.ec
   Read the corollary expanding the successful suffix, offset/size and prefix frame.

5. RANS_DECODER_INVARIANT.md
   Relate the pure state/cursor/segment coordinates to the actual outer/inner invariants.

6. Mode2RansDecoderActualTrace.ec
   Read the exact input predicate, $x = 2^{23}$ at loop exit, and the exposed postcondition.

7. Mode2RansDecoderTopHoare.ec
   Inspect the theorem directly opening the actual generated decoder.

8. In Mode2RansCoreCompositionBridge.ec, read the bridge transporting encoder/copy conclusions to the decoder input predicate.

9. In Mode2RansCoreActualInverse.ec, read the success-conditioned inverse of the three actual calls.

10. HBZ_FULL_WRAPPER_COMPOSITION.md
    In Mode2HbzFullEncodeTrace.ec and Mode2HbzFullDecodeInverse.ec, compare prepare/rANS/apply premise transport and wrapper-level size/frame.

11. Mode2HbzFullActualInverse.ec
    Read the failure/success disjunction, and in Mode2HbzSignatureBoundaryLift.ec check exact equivalence of the focused and production harnesses.

12. RANS_ACTUAL_SUCCESS_WITNESS.md
    In Mode2RansAllSixBudget.ec and Mode2RansActualSuccessWitness.ec, follow the 1020-byte budget, failure-cause contradiction and production zero round trip.

13. `WEEK16_MINCORE_PLAN.md`
`WEEK16_KG_REPORT.md`,
`WEEK16_KG_NTT_MUL_REPORT.md`,
`WEEK16_SIGN_REPORT.md`,
`WEEK16_VERIFY_REPORT.md`,
`Mode2KeygenSnapshotAlgebra.ec` and
`Mode2KeygenCoreEquation.ec`,
`Mode2KeygenNttMulBridge.ec`, and the five Sign/Verify targets.
Compare these files and separate the frozen KG-2-like/Sign-control/Verify helper-local boundaries from the three STOP verdicts.

14. `NTTFullSpectralAction.ec`, `NTTMode2RowSpecializations.ec`, `RqHAETAEBridge.ec`: follow the pure NTT, native R_q, and HAETAE-representation arrows closed after the freeze.

15. Compare `POST_FREEZE_KG_ACTUAL_AVEC_QJ_REPORT.md`, the three compiled `KgActualAvecQj*.ec` files and the deterministic trace checker. Keep actual `avec`, paper a, paper qj, and the synthetic security object distinct, and check `STOP-KG-AVEC-QJ`. Distinguish the freshly compiled actual-carrier matrix/secret-only structured coefficient-product in `KgFaithfulAugmentedModel.ec` from the still-missing generic security-model `matrix_vec_mul`/ExpandVecA edge.

16. Preserve the full verifier baseline before making changes.

Final reading rule

Read a proposition's actual procedure surface, premises and postcondition, not just its theorem name or filename. "Pure inverse," "actual control," "actual refinement," and "full parent" are four different claims.

References

[1] CryptoLab. HAETAE specification, 2026. Pinned local artifact CryptoLab_HAETAE.pdf, 46 pages, produced 2026-02-04; SHA-256 80d28b3af9af2a27dba0eb4746f9b5755e05cb9a86bb6596f45fda06c605f3f6.

[2] CryptoLab. Pinned HAETAE jasmin implementation tree, 2026. Local source root haetae-ref-jasmin, used by focused jasmin2ec extraction.

[3] HAETAE top-down EasyCrypt project. Claim ledger, 2026. Operational source of truth, local document haetae-topdown-easycrypt/CLAIM_LEDGER.md, Week 16 snapshot.

[4] HAETAE top-down EasyCrypt project. Frozen paper artifact manifest, 2026. Local document haetae-topdown-easycrypt/manifests/paper-artifacts.md.

[5] HAETAE top-down EasyCrypt project. Mode-2 actual rans core composition, 2026. Local document haetae-topdown-easycrypt/RANS_CORE_COMPOSITION.md.

[6] HAETAE top-down EasyCrypt project. Mode-2 actual rans decoder invariant, 2026. Local document haetae-topdown-easycrypt/RANS_DECODER_INVARIANT.md.

[7] HAETAE top-down EasyCrypt project. Mode-2 actual rans encoder invariant, 2026. Local document haetae-topdown-easycrypt/RANS_ENCODER_INVARIANT.md.

[8] HAETAE top-down EasyCrypt project. Mode-2 actual rans fixed-input success witness, 2026. Local document haetae-topdown-easycrypt/RANS_ACTUAL_SUCCESS_WITNESS.md.

[9] HAETAE top-down EasyCrypt project. Mode-2 full HBZ wrapper composition, 2026. Local document haetae-topdown-easycrypt/HBZ_FULL_WRAPPER_COMPOSITION.md.

[10] HAETAE top-down EasyCrypt project. Post-freeze actual avec and paper a,qj report, 2026. Local document haetae-topdown-easycrypt/POST_FREEZE_KG_ACTUAL_AVEC_QJ_REPORT.md.

[11] HAETAE top-down EasyCrypt project. Post-freeze faithful augmented KeyGen model report, 2026. Local document haetae-topdown-easycrypt/POST_FREEZE_KG_FAITHFUL_AUGMENTED_MODEL_REPORT.md.

[12] HAETAE top-down EasyCrypt project. Post-freeze KG Rq/HAETAE bridge report, 2026. Local document haetae-topdown-easycrypt/POST_FREEZE_KG_RQ_HAETAE_REPORT.md.

[13] HAETAE top-down EasyCrypt project. Project scope and trust boundary, 2026. Pinned local document haetae-topdown-easycrypt/PROJECT_SCOPE.md.

[14] HAETAE top-down EasyCrypt project. Theorem graph, 2026. Local dependency graph haetae-topdown-easycrypt/THEOREM_GRAPH.md.

[15] HAETAE top-down EasyCrypt project. Week 10 report: Actual rans encoder inner loop and size bound, 2026. Local document haetae-topdown-easycrypt/WEEK10_REPORT.md.

[16] HAETAE top-down EasyCrypt project. Week 11 report: Actual rans encoder trace closure, 2026. Local document haetae-topdown-easycrypt/WEEK11_REPORT.md.

[17] HAETAE top-down EasyCrypt project. Week 12 report: Actual rans decoder semantic refinement, 2026. Local document haetae-topdown-easycrypt/WEEK12_REPORT.md.

[18] HAETAE top-down EasyCrypt project. Week 13 report: Actual encoder/copy/decoder core inverse, 2026. Local document haetae-topdown-easycrypt/WEEK13_REPORT.md.

[19] HAETAE top-down EasyCrypt project. Week 14 report: Production full HBZ wrapper boundary, 2026. Local document haetae-topdown-easycrypt/WEEK14_REPORT.md.

[20] HAETAE top-down EasyCrypt project. Week 15 report: Actual all-6 success witness, 2026. Local document haetae-topdown-easycrypt/WEEK15_REPORT.md.

[21] HAETAE top-down EasyCrypt project. Week 16 KG–NTT–MUL continuation report, 2026. Local document haetae-topdown-easycrypt/WEEK16_KG_NTT_MUL_REPORT.md.

[22] HAETAE top-down EasyCrypt project. Week 16 mincore keygen first-task report, 2026. Local document haetae-topdown-easycrypt/WEEK16_KG_REPORT.md.

[23] HAETAE top-down EasyCrypt project. Week 16 mincore keygen snapshot plan, 2026. Local document haetae-topdown-easycrypt/WEEK16_MINCORE_PLAN.md.

[24] HAETAE top-down EasyCrypt project. Week 16 MINCORE Sign report, 2026. Local document haetae-topdown-easycrypt/WEEK16_SIGN_REPORT.md.

[25] HAETAE top-down EasyCrypt project. Week 16 MINCORE Verify and matrix/CRT report, 2026. Local document haetae-topdown-easycrypt/WEEK16_VERIFY_REPORT.md.

[26] HAETAE top-down EasyCrypt project. Week 7 product/replay decision, 2026. Local document haetae-topdown-easycrypt/PRODUCT_REPLAY_DECISION.md.